CS11313 - Fall 2024

Design & Analysis of Algorithms

Graph Algorithms: **Topological Sort**

Ibrahim Albluwi

Motivation: Cell Evaluation in Spreadsheets

Problem. In what order should the cells be evaluated?

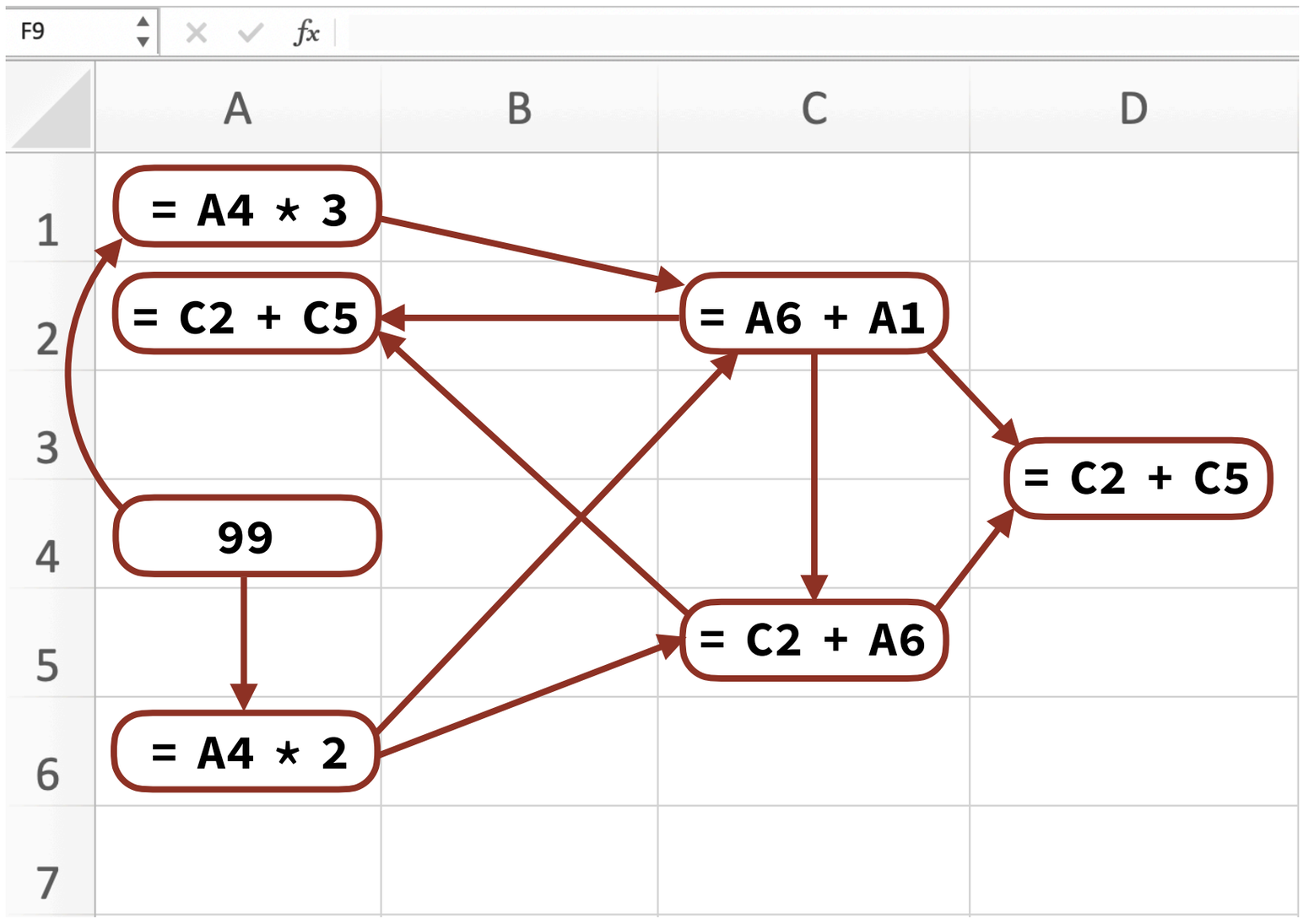
F9 fx

	A	B	C	D
1	= A4 * 3			
2	= C2 + C5		= A6 + A1	
3				= C2 + C5
4	99			
5			= C2 + A6	
6	= A4 * 2			
7				

Motivation: Cell Evaluation in Spreadsheets

Problem. In what order should the cells be evaluated?

Dependency Graph. Edge $u \to v$ means that vertex v depends on vertex u.



Must be a **DAG**

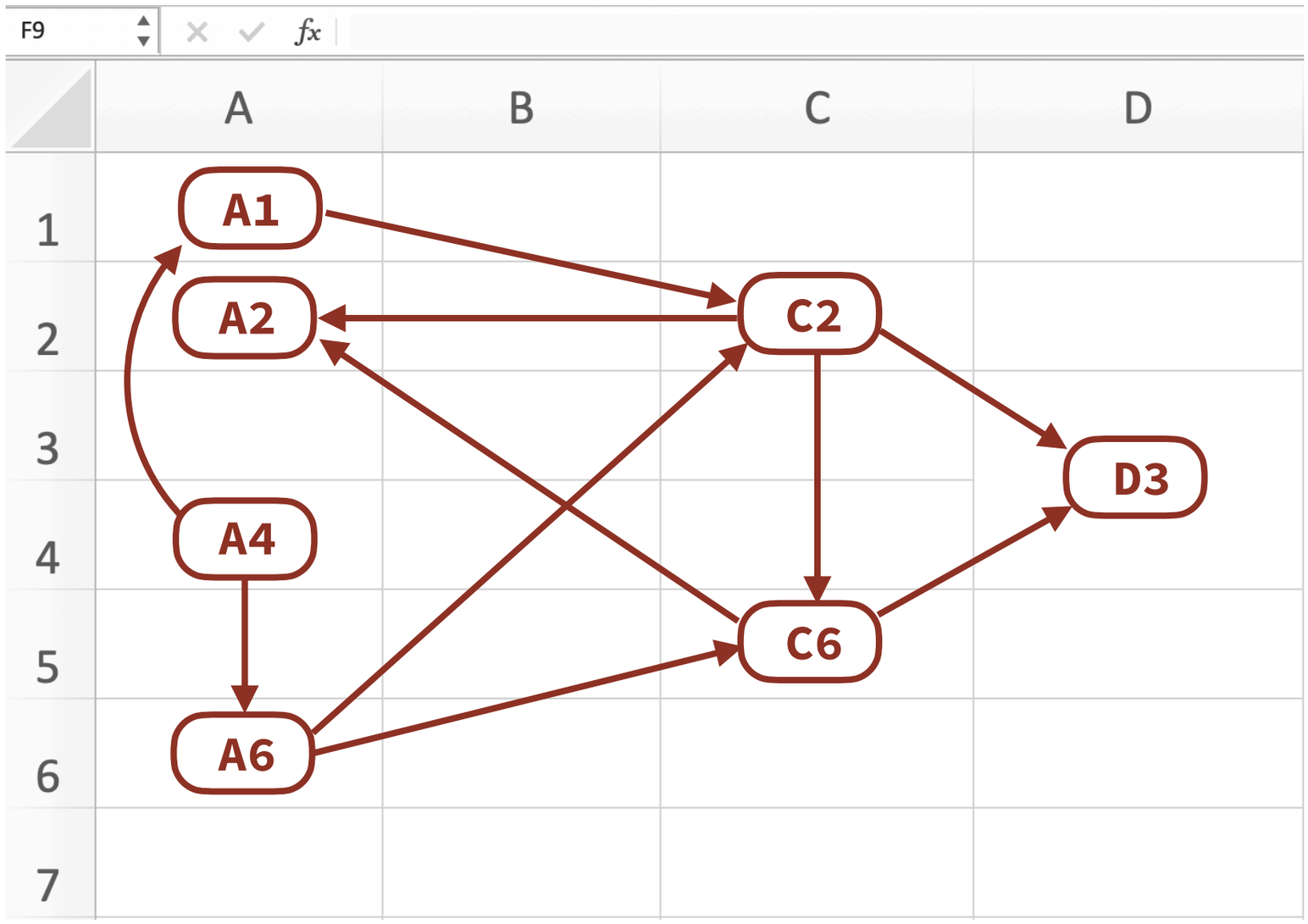
(cyclic dependencies are not allowed!)

Motivation: Cell Evaluation in Spreadsheets

Problem. In what order should the cells be evaluated?

Dependency Graph. Edge $u \to v$ means that vertex v depends on vertex u.

Example. We can't evaluate **D3** until **C2** and **C5** are evaluated!



Must be a **DAG**

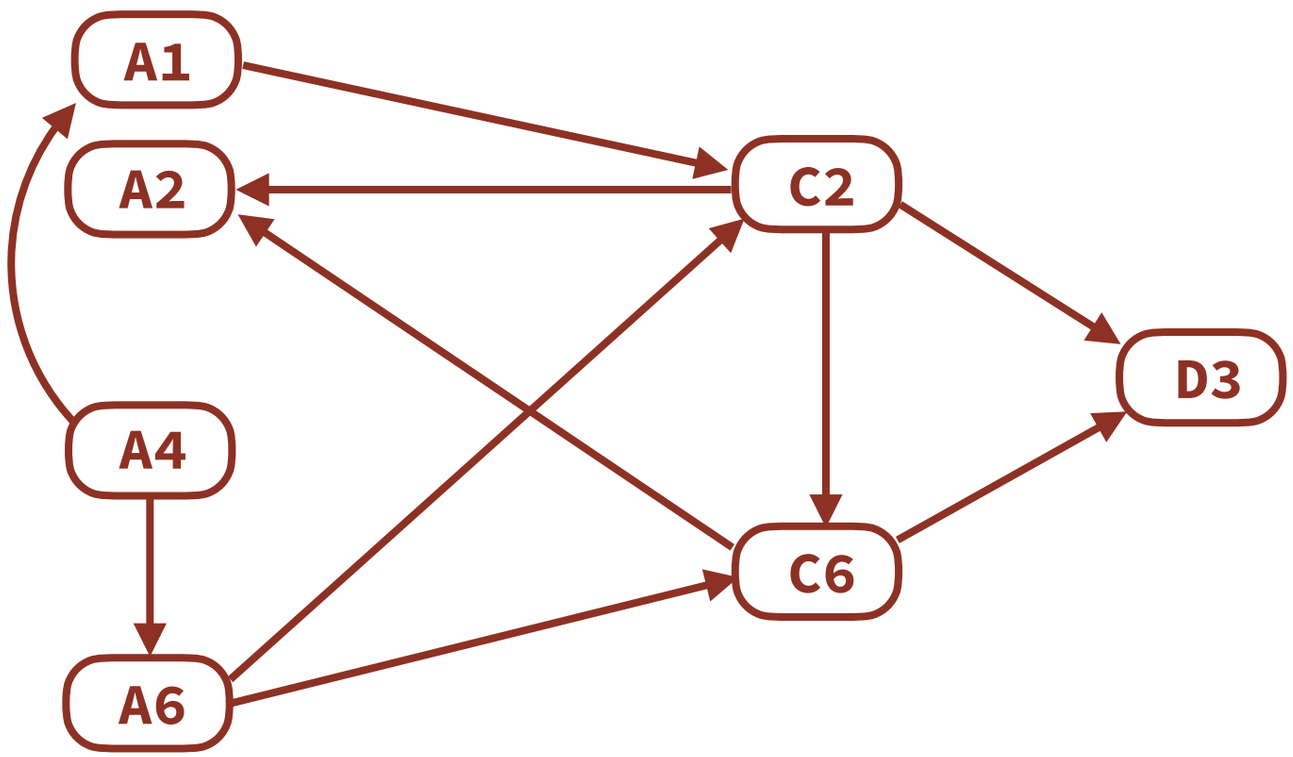
(cyclic dependencies are not allowed!)

Motivation: Cell Evaluation in Spreadsheets

Problem. In what order should the cells be evaluated?

Dependency Graph. Edge $u \to v$ means that vertex v depends on vertex u.

Example. We can't evaluate **D3** until **C2** and **C5** are evaluated!



Must be a **DAG**

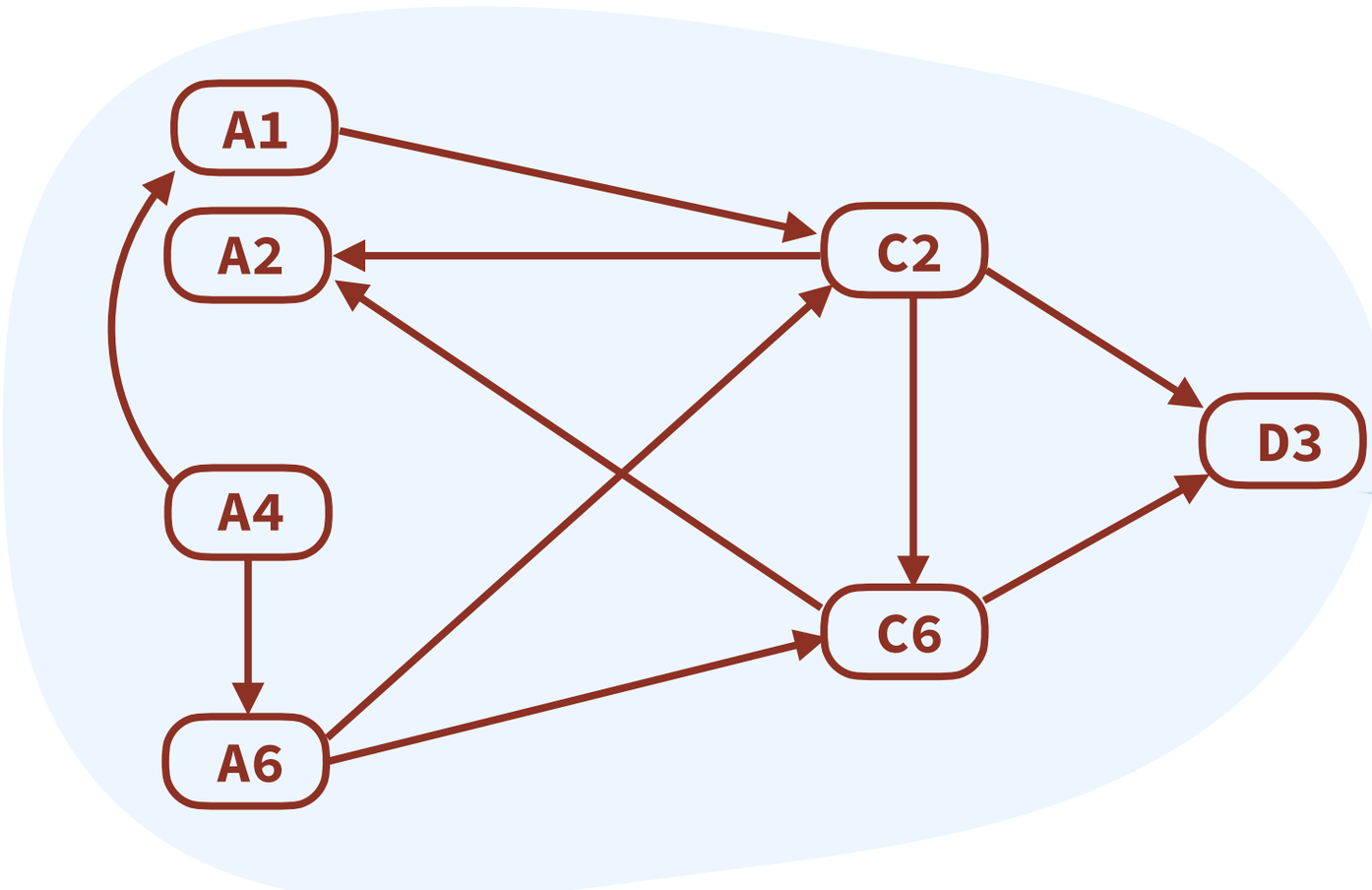
(cyclic dependencies are not allowed!)

Motivation: Cell Evaluation in Spreadsheets

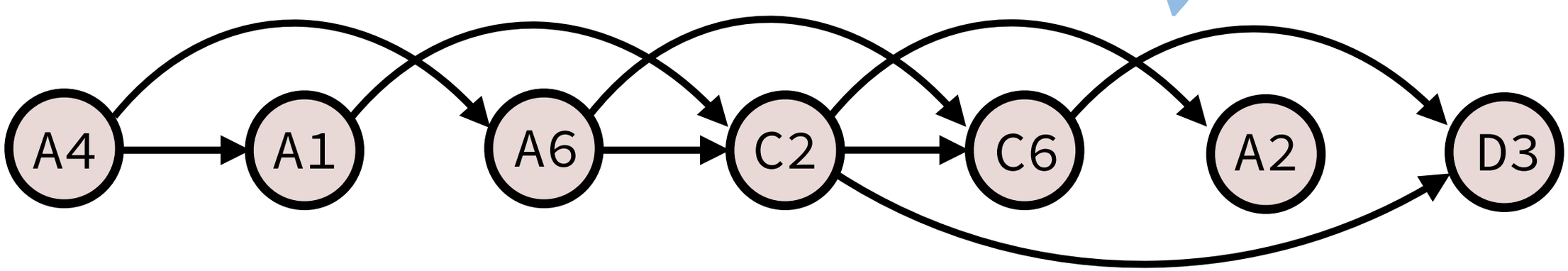
Problem. In what order should the cells be evaluated?

Dependency Graph. Edge $u \to v$ means that vertex v depends on vertex u.

Example. We can't evaluate **D3** until **C2** and **C6** are evaluated!



An **isomorphic** graph
(same graph drawn differently)



Motivation: Cell Evaluation in Spreadsheets

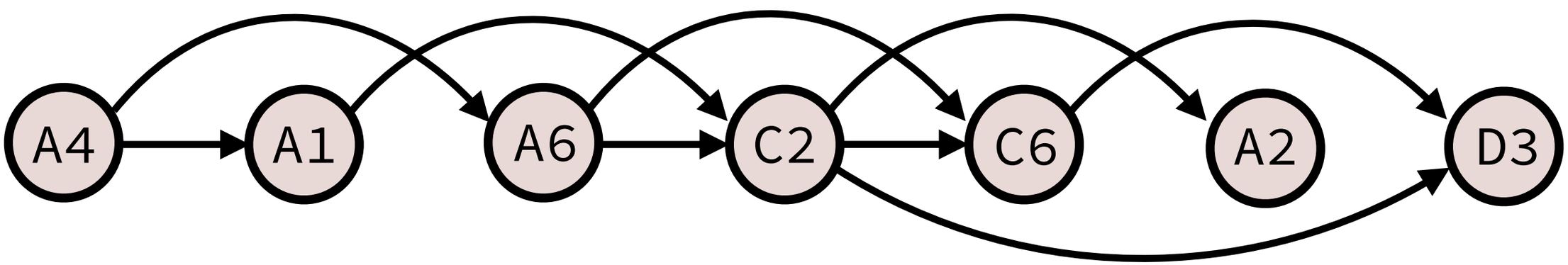
Problem. In what order should the cells be evaluated?

Dependency Graph. Edge $u \to v$ means that vertex v depends on vertex u.

Example. We can't evaluate **`D3`** until **`C2`** and **`C6`** are evaluated!

Valid Orderings.

```
A4, A1, A6, C2, C6, A2, D3
A4, A6, A1, C2, C6, A2, D3
A4, A6, A1, C2, C6, D3, A2
```

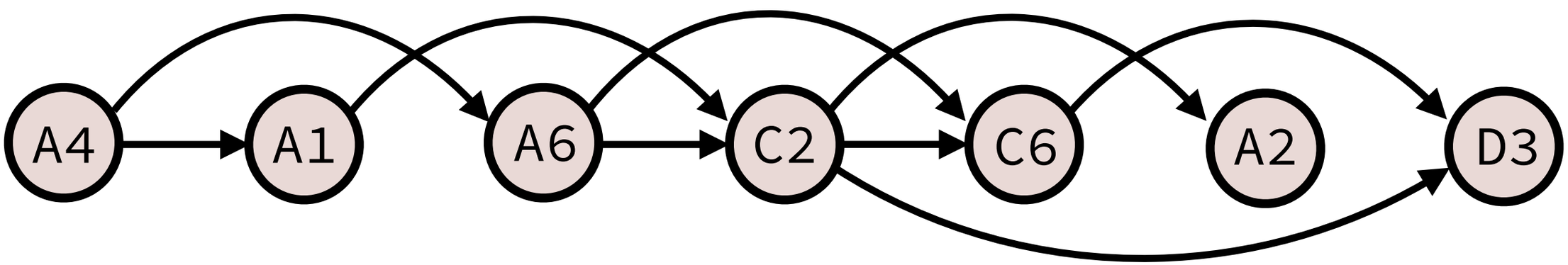


Topological Sort: Definition

Problem. Given a directed acyclic graph G, provide an ordering for the vertices, such that if there is an edge $(u \to v)$ in G, then u appears before v in the ordering.

Valid Topological Sorts.

```
A4, A1, A6, C2, C6, A2, D3
A4, A6, A1, C2, C6, A2, D3
A4, A6, A1, C2, C6, D3, A2
```



Topological Sort: Example Applications

PERT Charts. Given a graph representing task dependencies in a software engineering project, provide an ordering for task completion that respects these dependencies.

Topological Sort: Example Applications

PERT Charts. Given a graph representing task dependencies in a software engineering project, provide an ordering for task completion that respects these dependencies.

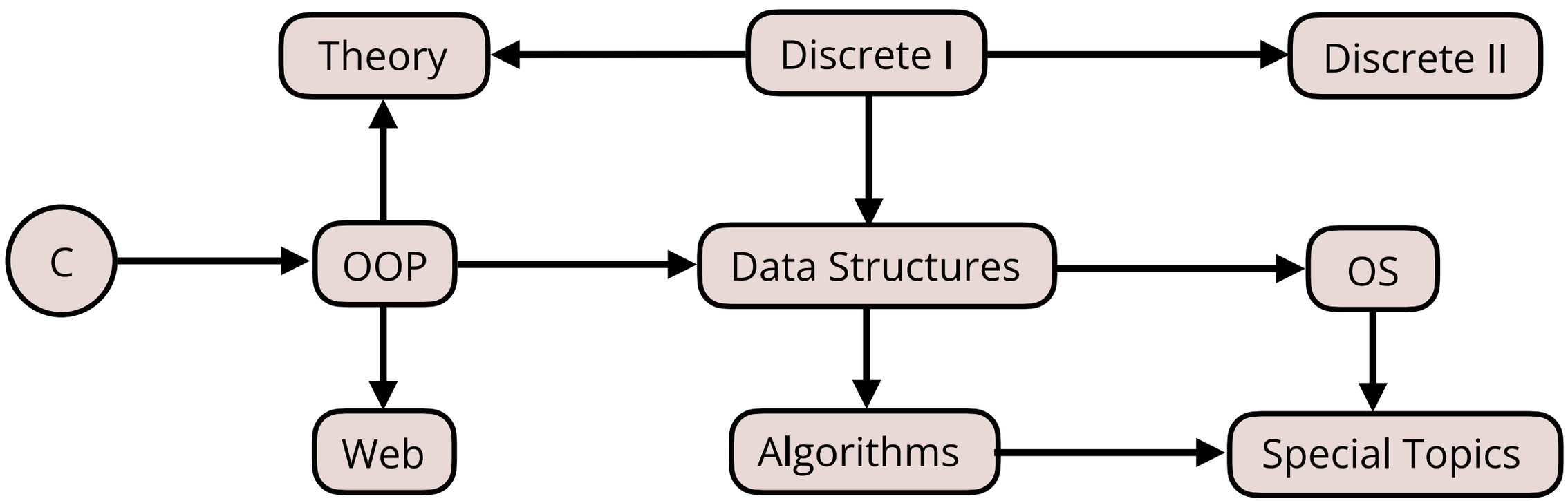
Compilation Order. Given a set of source files with dependencies (e.g. **`file1`** imports **`file2`**, etc.). In what order should the files be compiled?

Topological Sort: Example Applications

PERT Charts. Given a graph representing task dependencies in a software engineering project, provide an ordering for task completion that respects these dependencies.

Compilation Order. Given a set of source files with dependencies (e.g. **`file1`** imports **`file2`**, etc.). In what order should the files be compiled?

Course Scheduling. Given a DAG representing course prerequisite relationships, provide an ordering for the courses such that a course in the ordering always appears after all of its prerequisites.

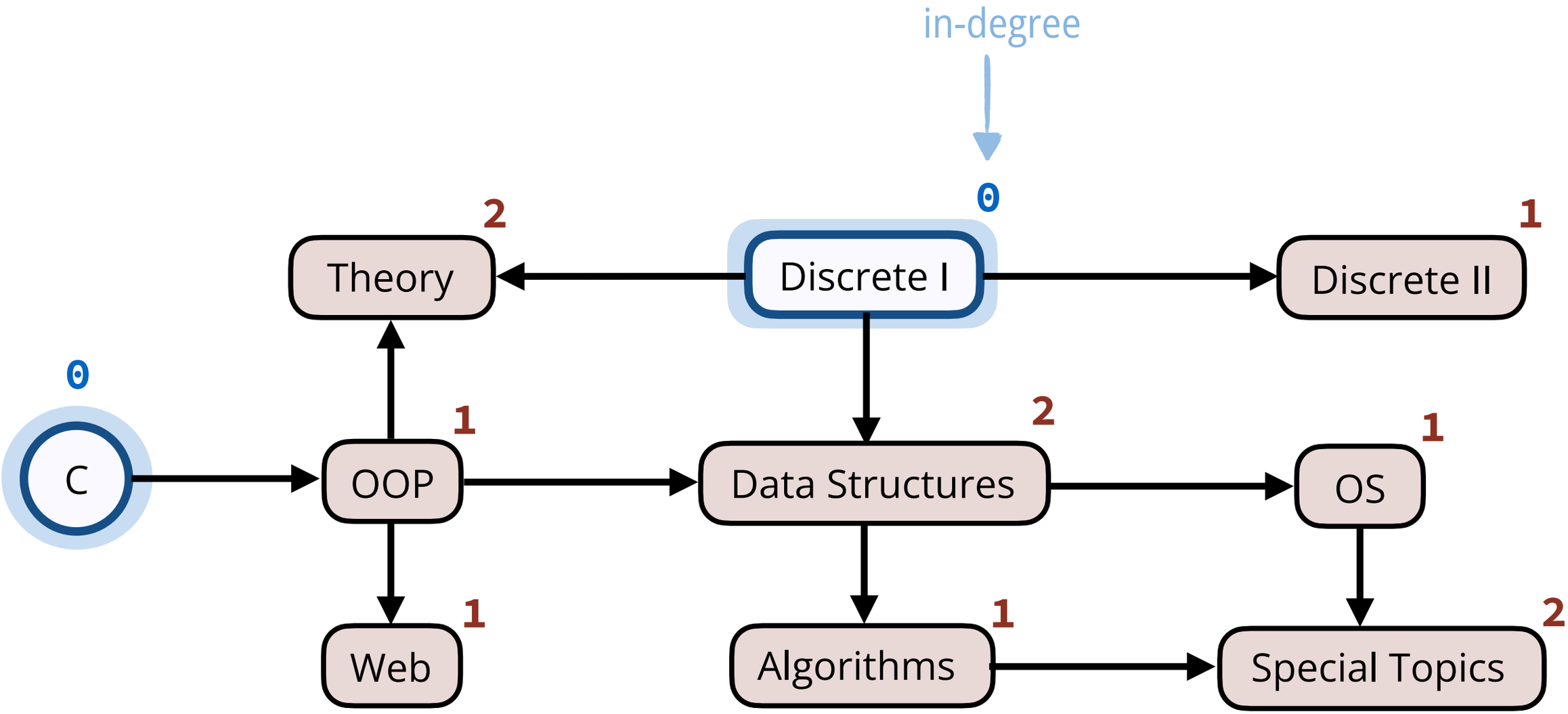


Topological Sort: Algorithm # 1

optional

Main Idea. For each vertex v with in-degree `0`:

1. Print the vertex.
2. Subtract **1** from the in-degree of every vertex w with an edge $(v \rightarrow w)$.
3. remove the vertex.



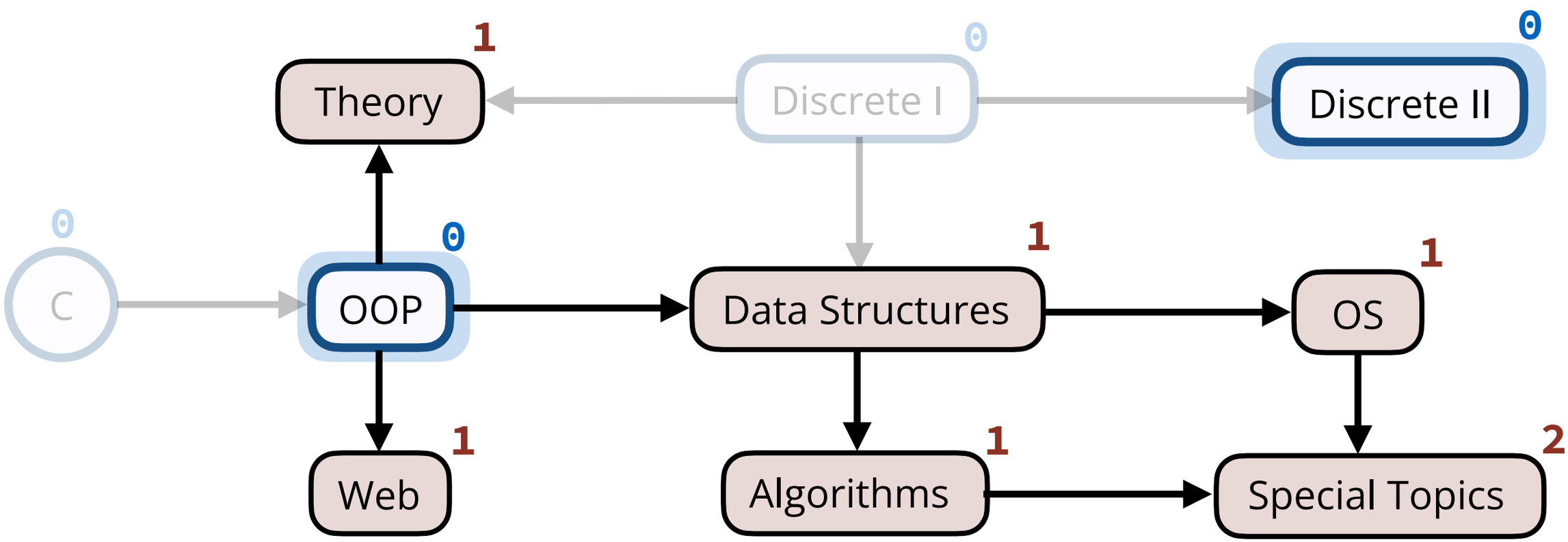
Topological Sort: Algorithm # 1

optional

Main Idea. For each vertex v with in-degree `0`:

1. Print the vertex.
2. Subtract **1** from the in-degree of every vertex w with an edge $(v \to w)$.
3. remove the vertex.

Topological Sort: C → Discrete I →



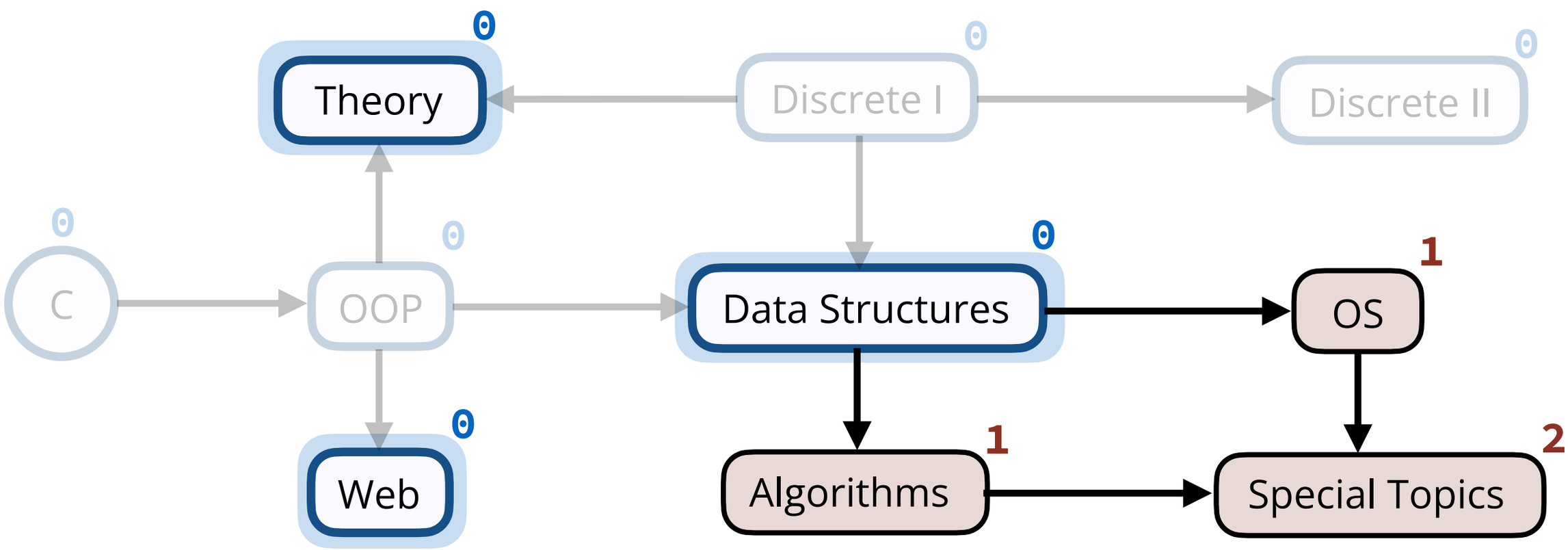
Topological Sort: Algorithm # 1

optional

Main Idea. For each vertex v with in-degree `0`:

1. Print the vertex.
2. Subtract **1** from the in-degree of every vertex w with an edge $(v \to w)$.
3. remove the vertex.

Topological Sort: C → Discrete I → Discrete II → OOP



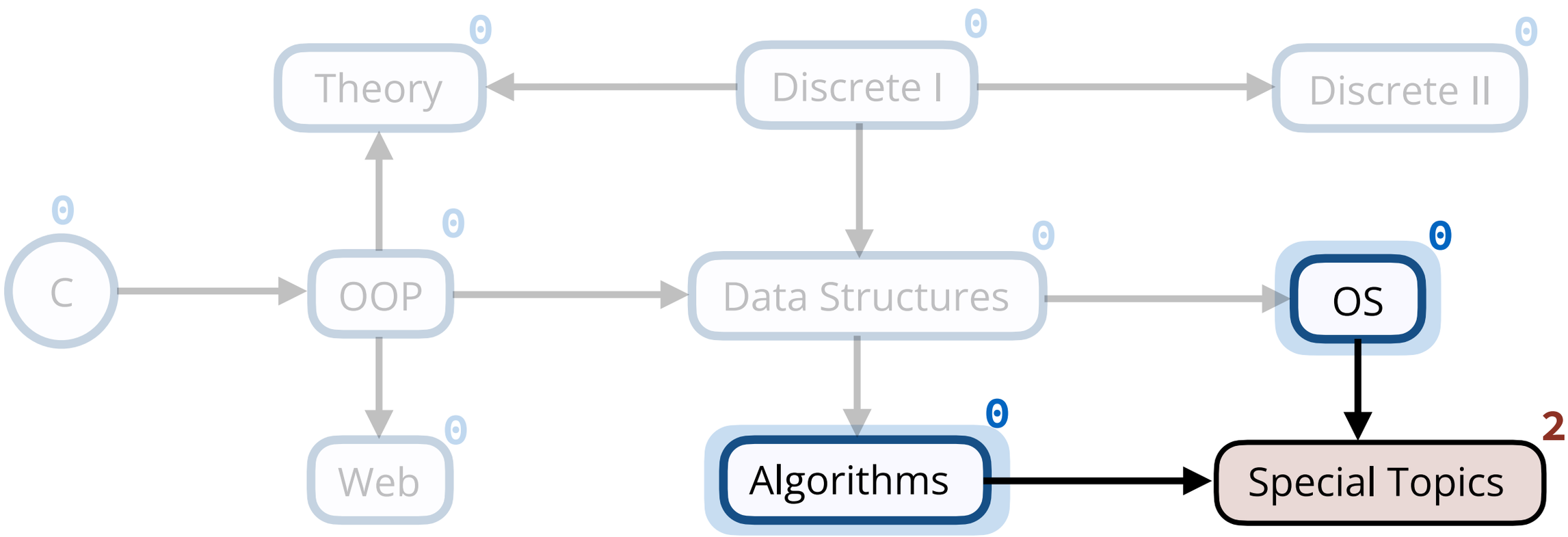
Topological Sort: Algorithm # 1

optional

Main Idea. For each vertex v with in-degree `0`:

1. Print the vertex.
2. Subtract **1** from the in-degree of every vertex w with an edge $(v \to w)$.
3. remove the vertex.

Topological Sort: C → Discrete I → Discrete II → OOP → Theory → Web → Data Structures



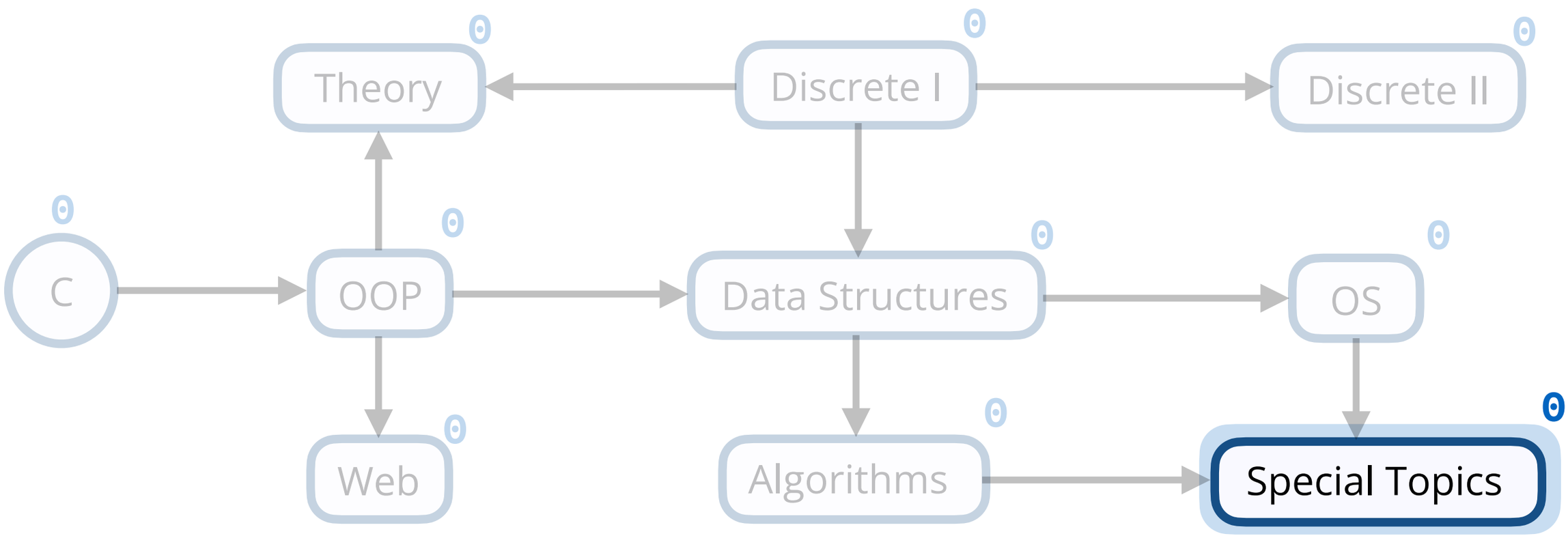
Topological Sort: Algorithm # 1

optional

Main Idea. For each vertex v with in-degree `0`:

1. Print the vertex.
2. Subtract **1** from the in-degree of every vertex w with an edge $(v \to w)$.
3. remove the vertex.

Topological Sort: C → Discrete I → Discrete II → OOP → Theory → Web → Data Structures → Algorithms → OS



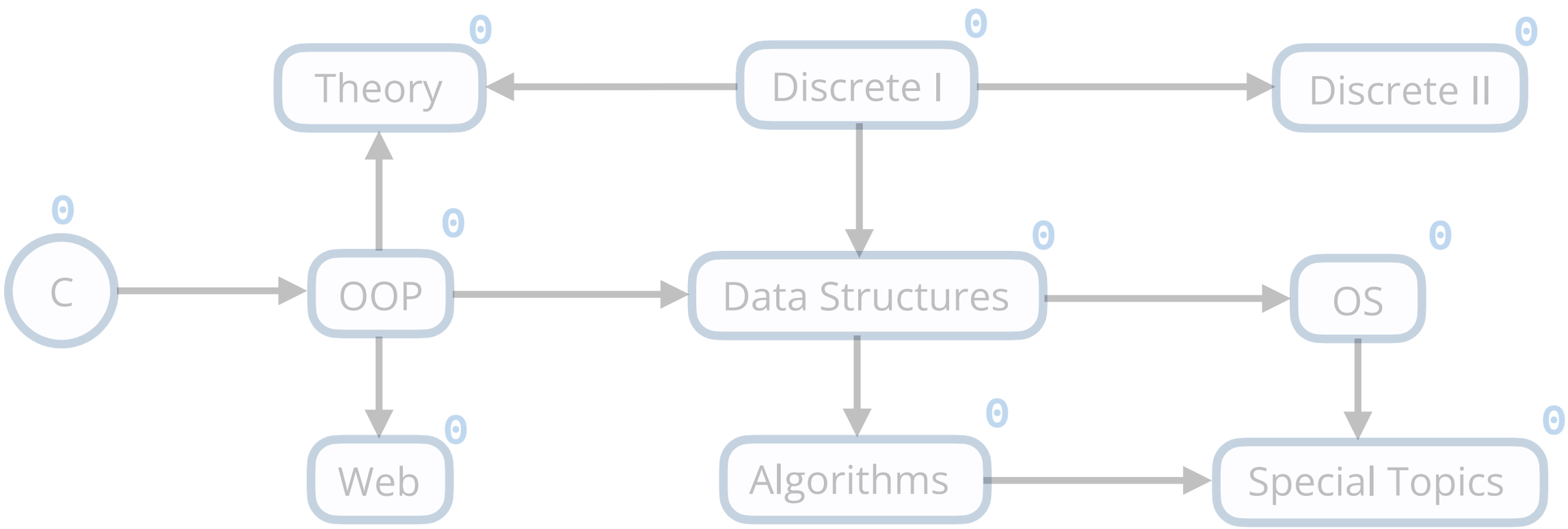
Topological Sort: Algorithm # 1

optional

Main Idea. For each vertex v with in-degree `0`:

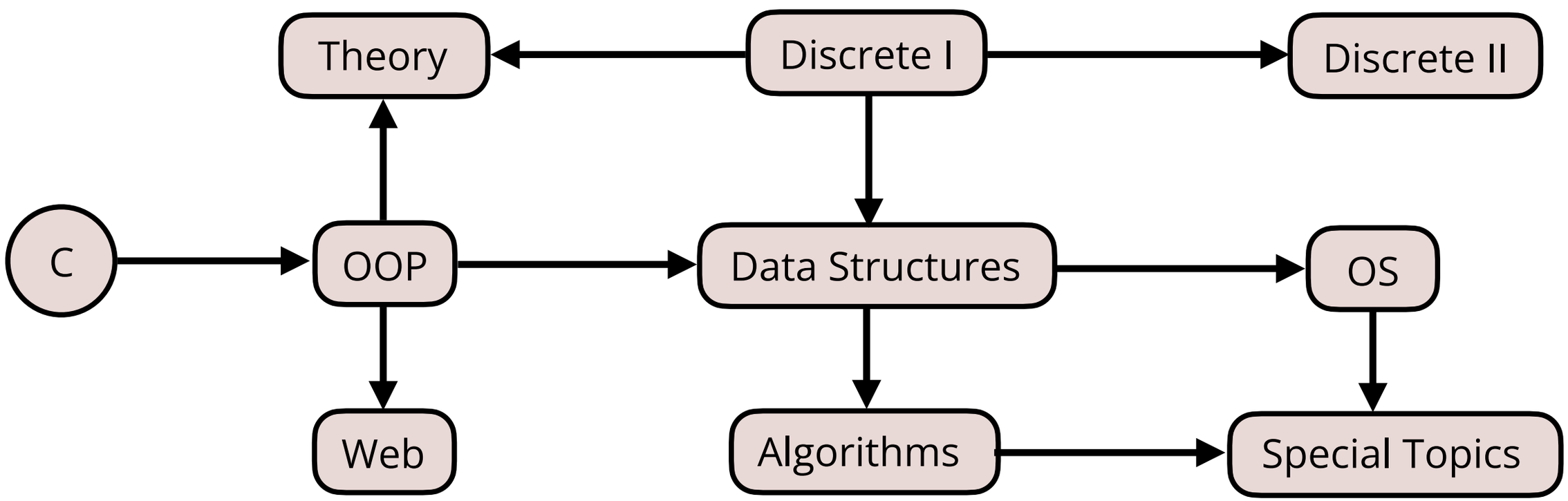
1. Print the vertex.
2. Subtract **1** from the in-degree of every vertex w with an edge $(v \to w)$.
3. remove the vertex.

Topological Sort: C → Discrete I → Discrete II → OOP → Theory → Web → Data Structures → Algorithms → OS → Special Topics



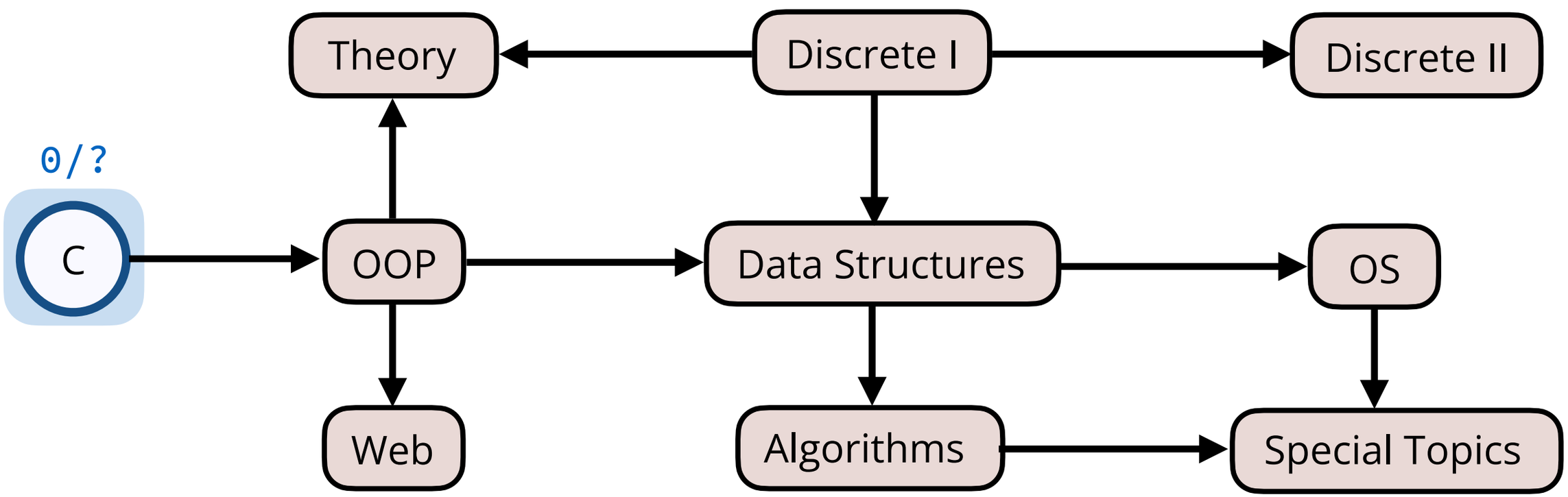
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



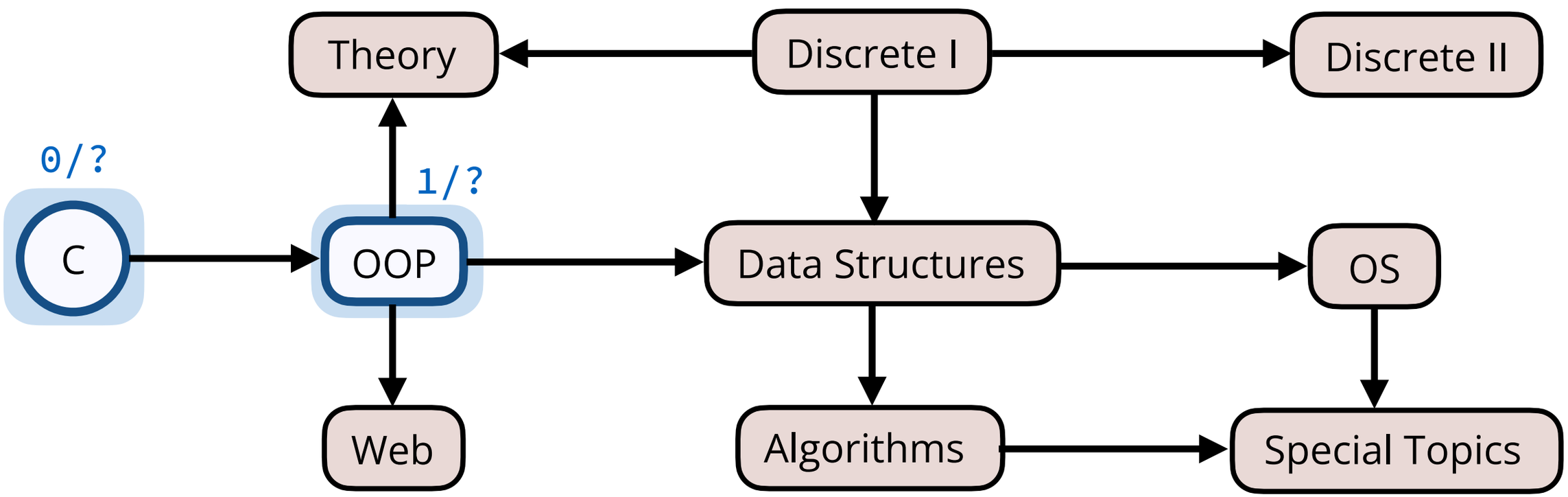
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



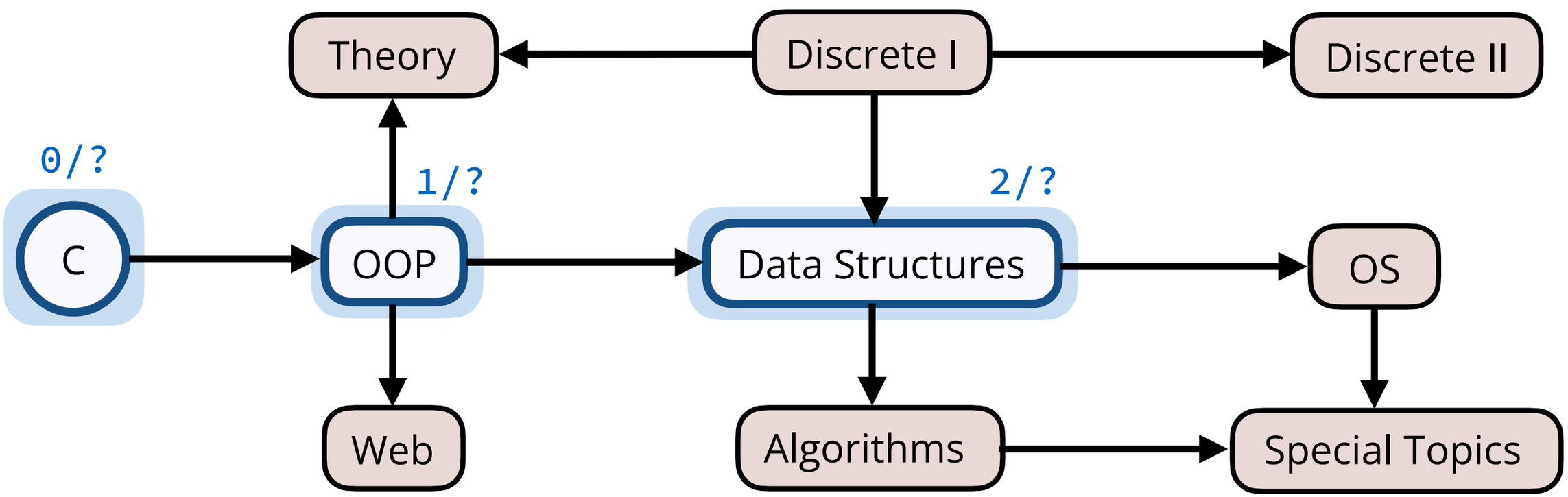
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



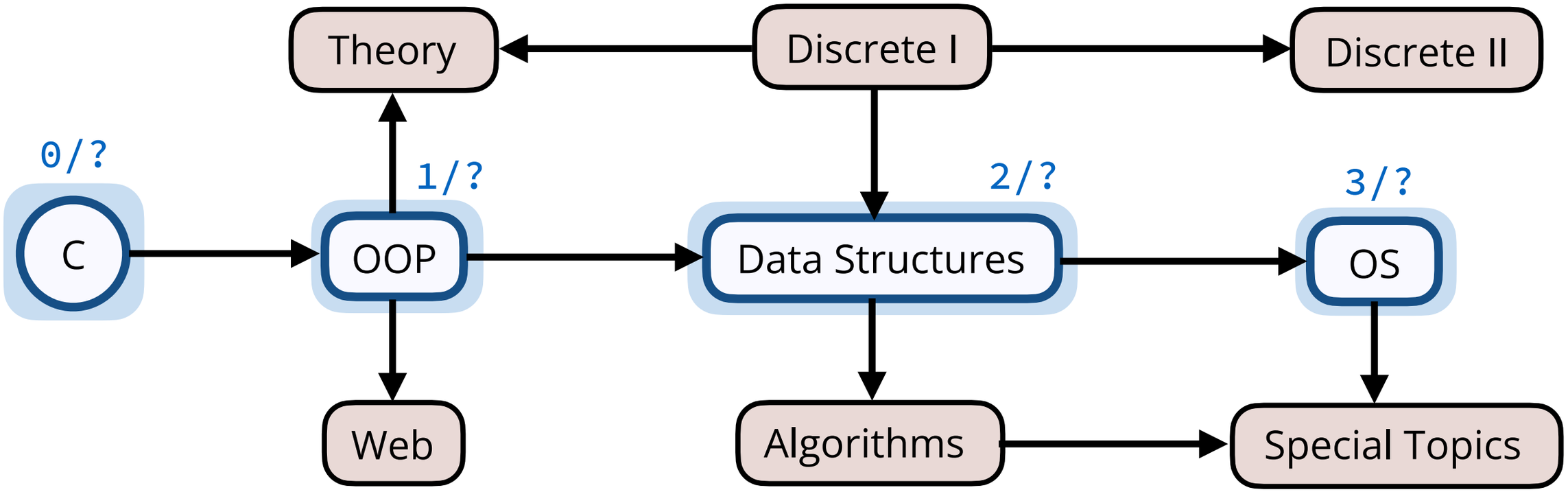
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



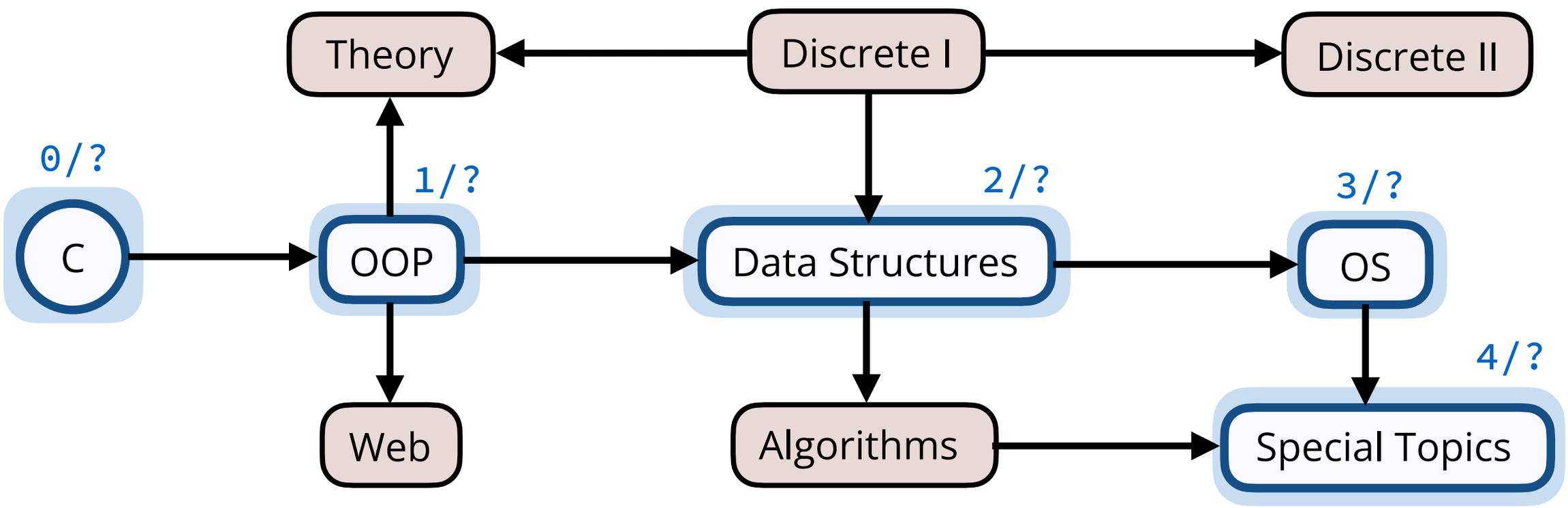
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



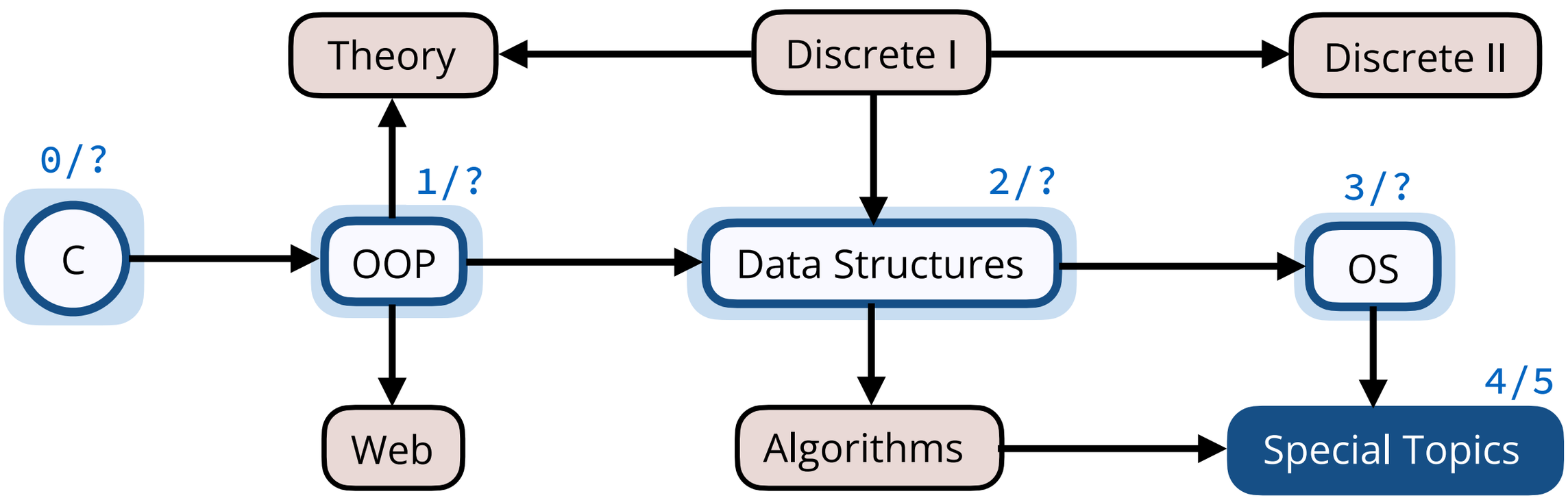
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



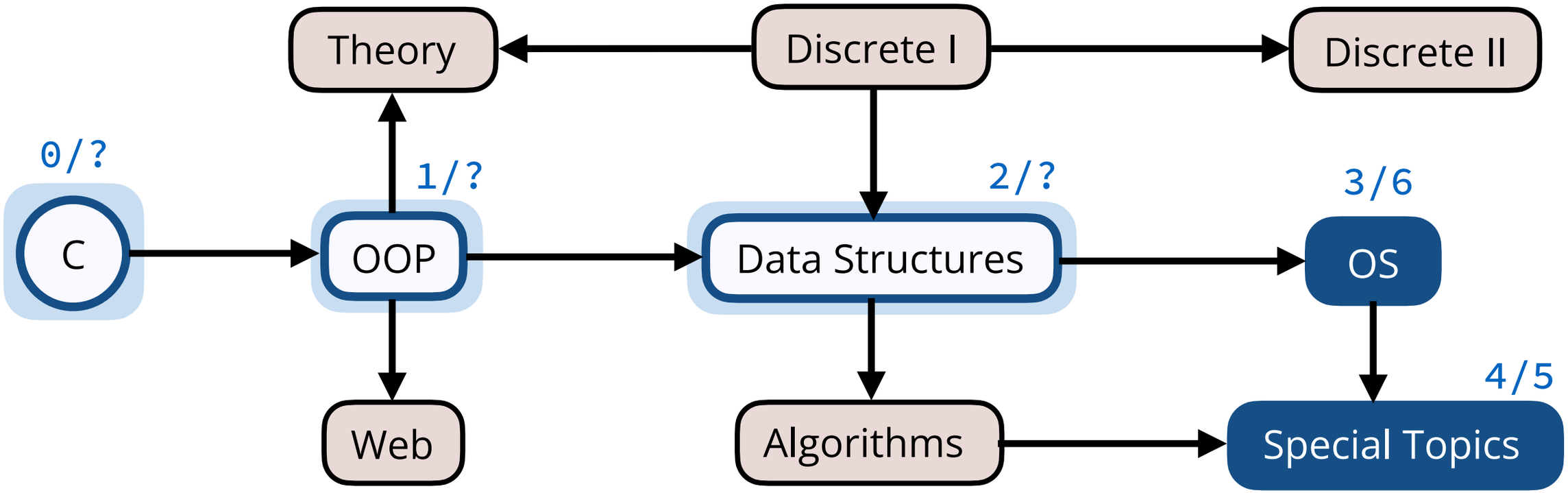
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



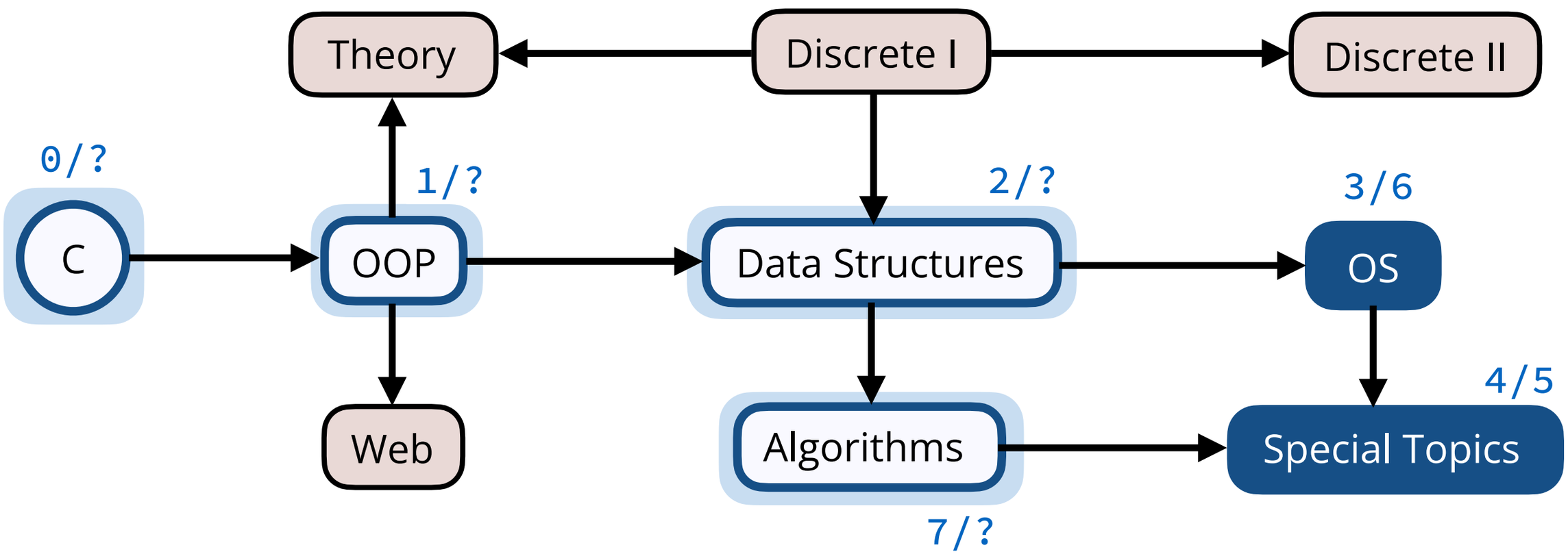
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



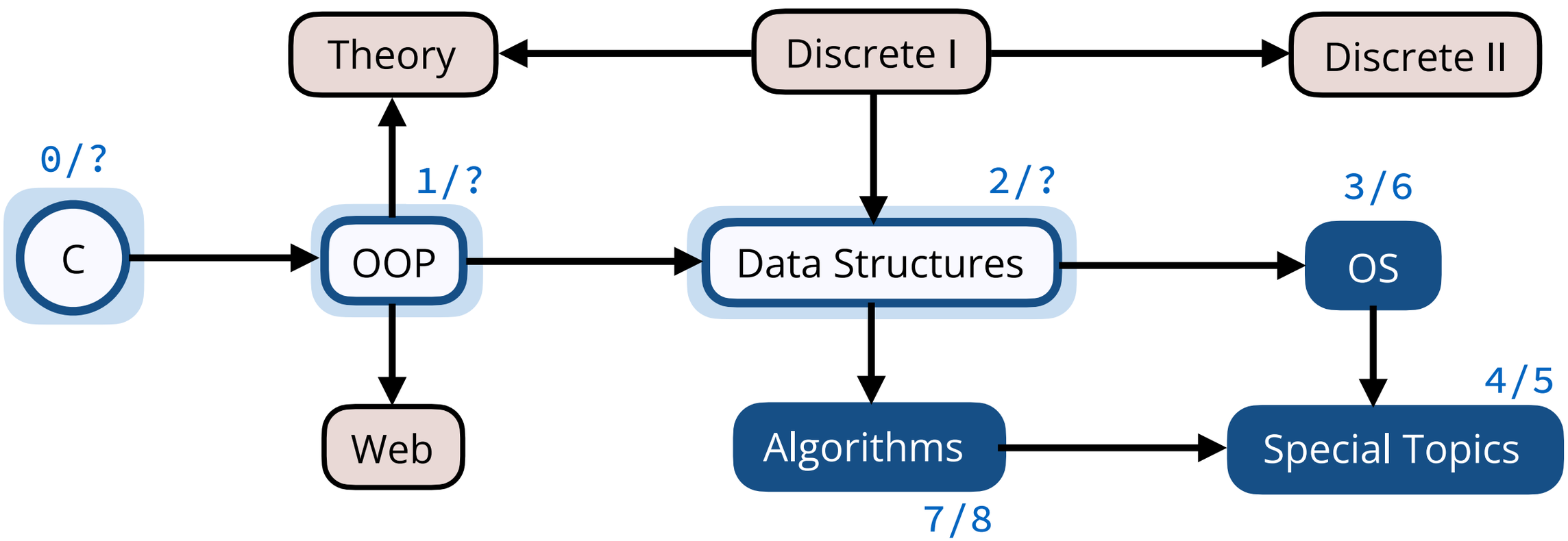
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



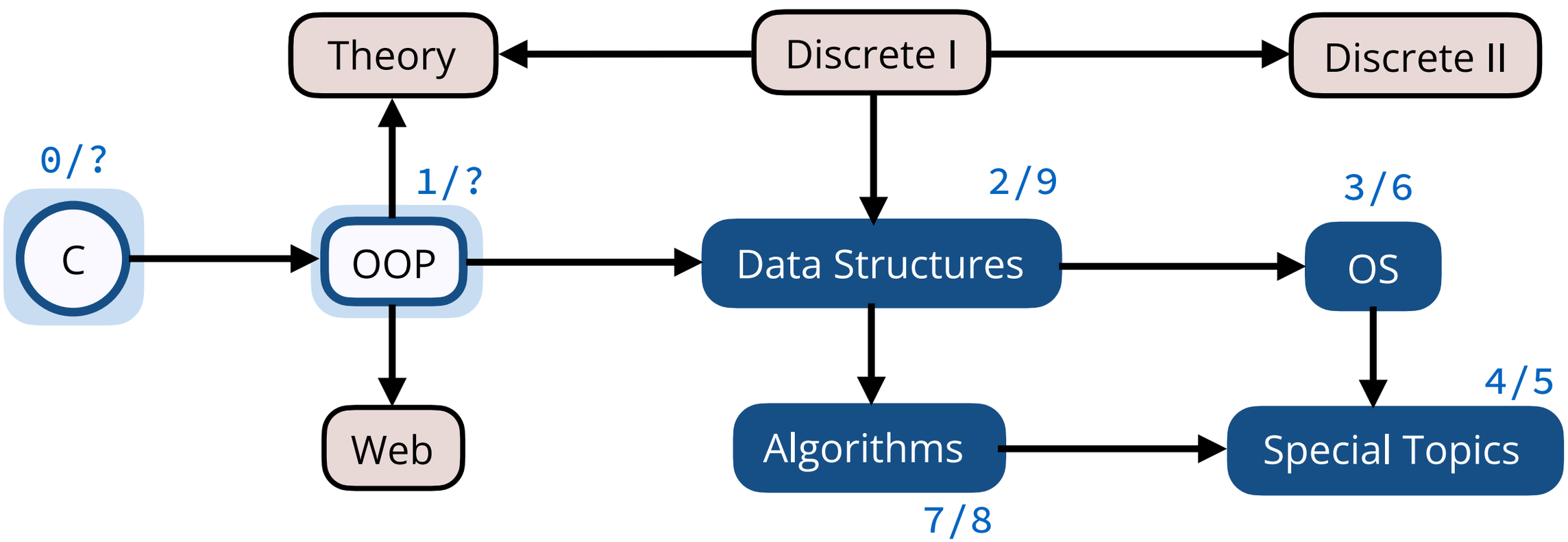
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



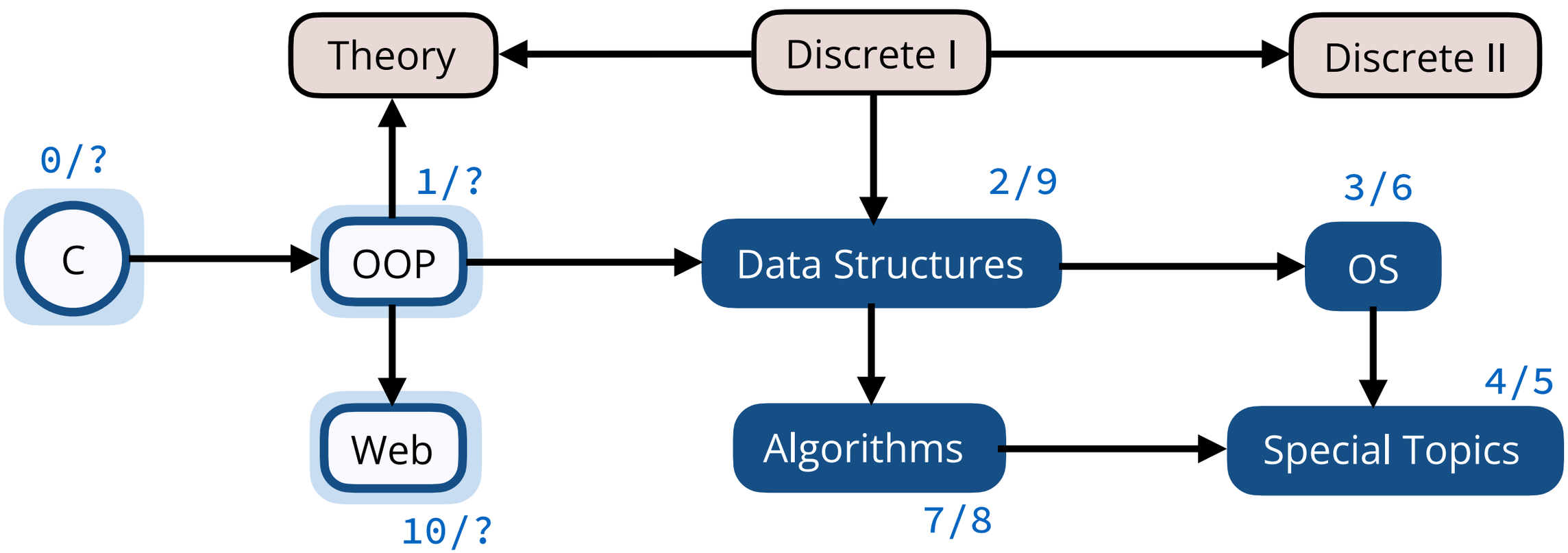
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



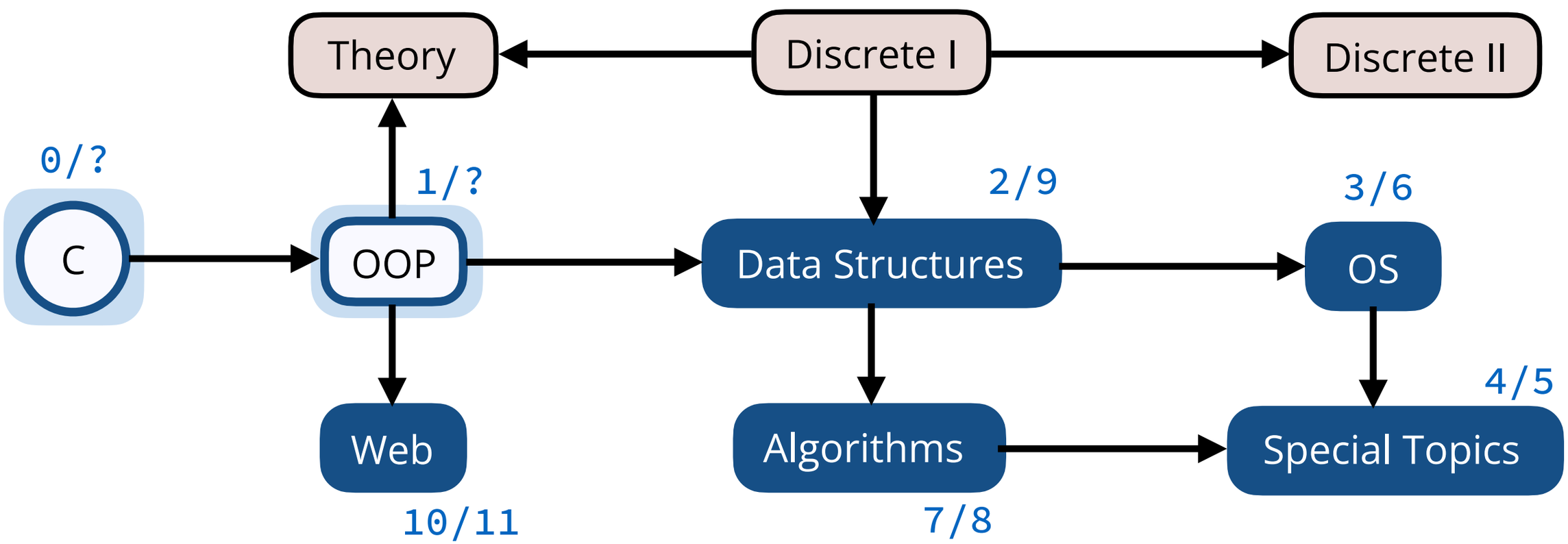
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



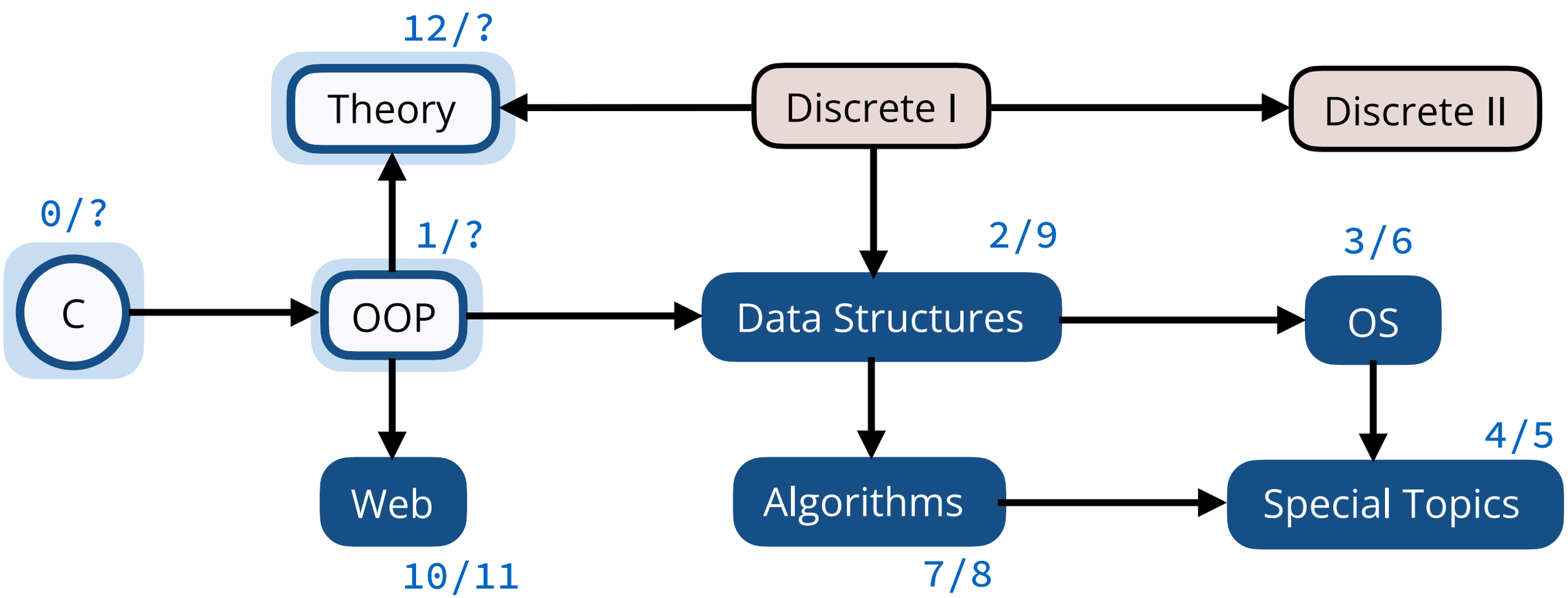
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



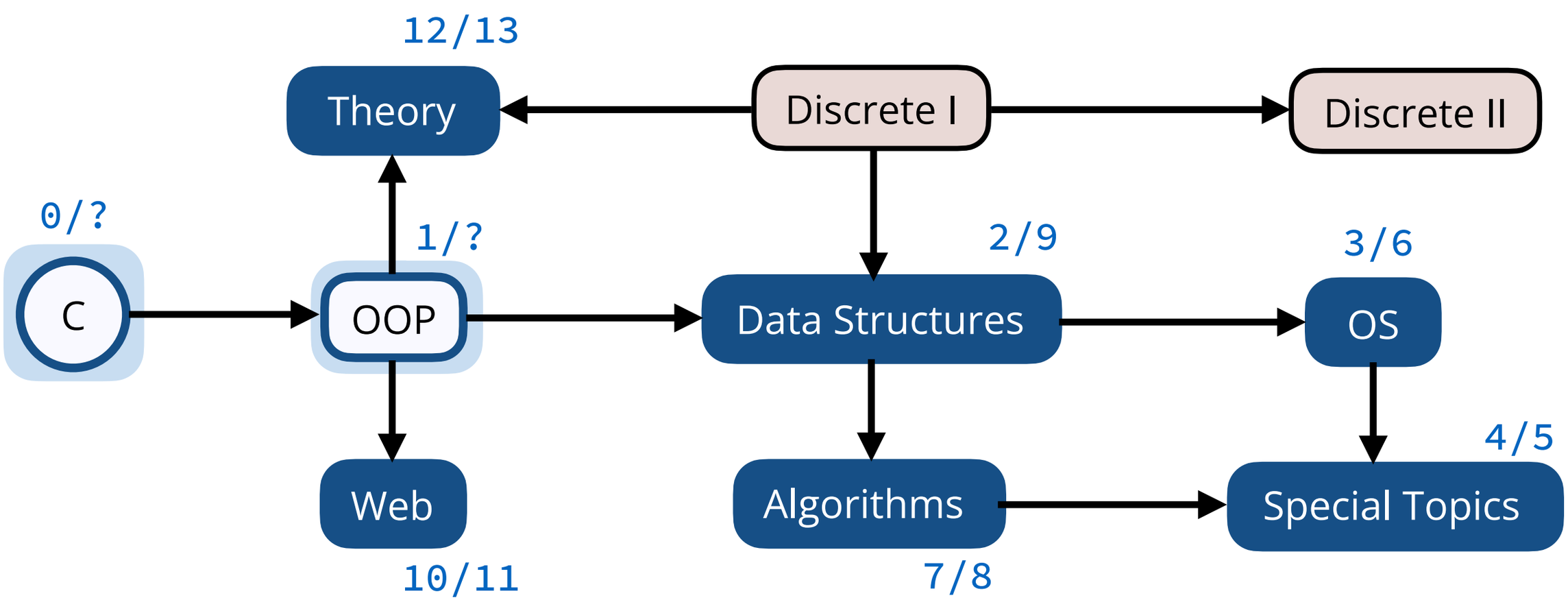
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



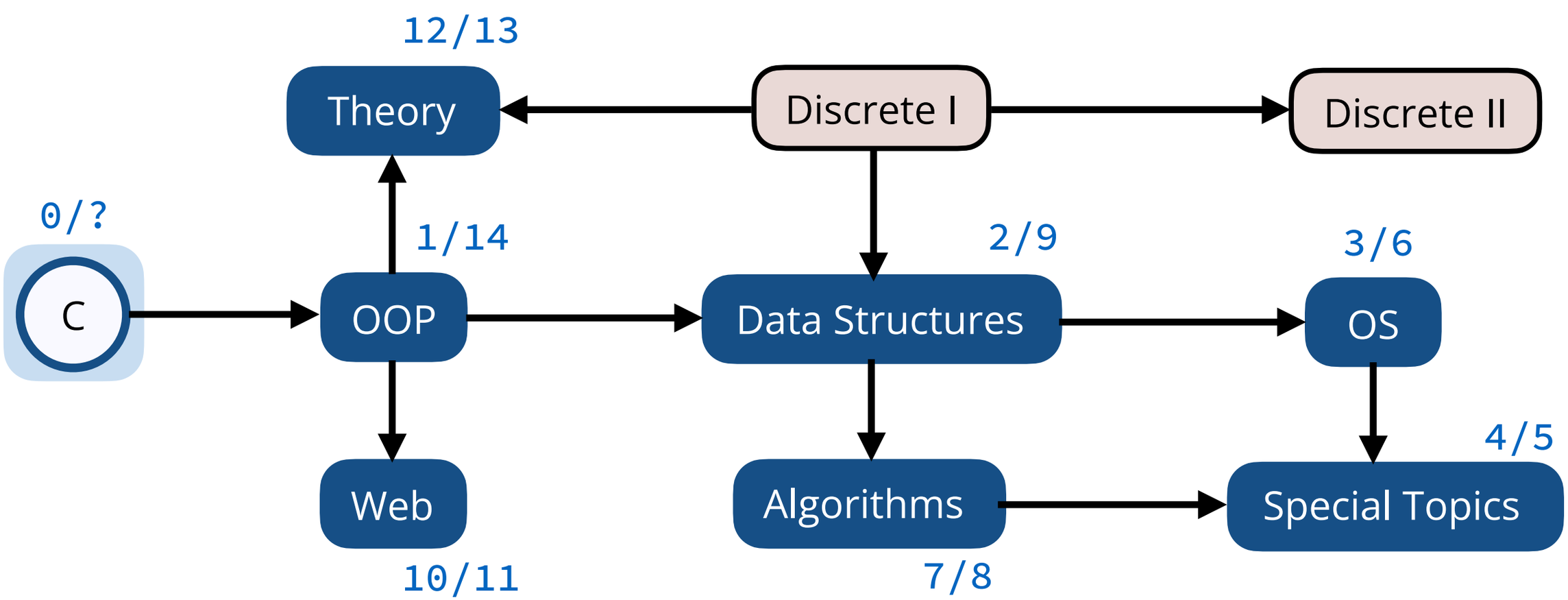
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



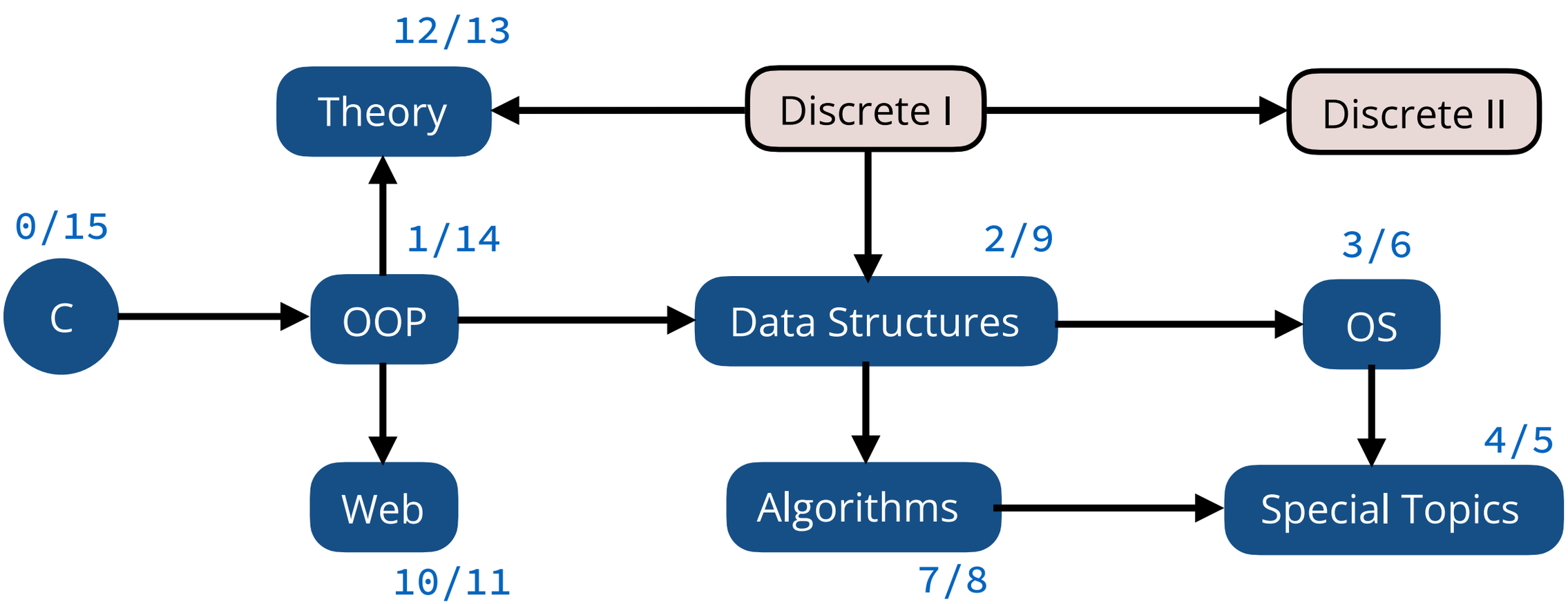
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



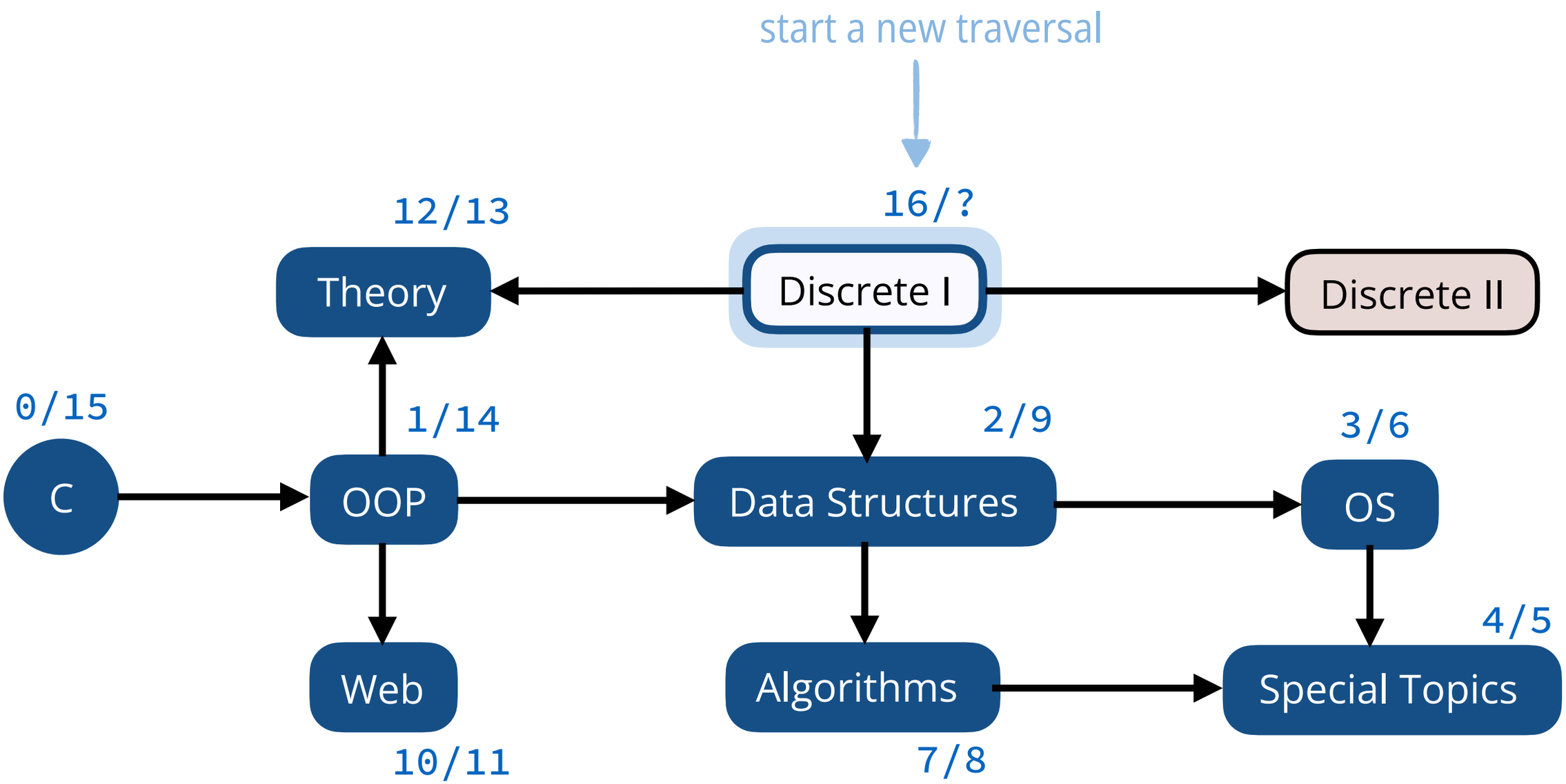
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



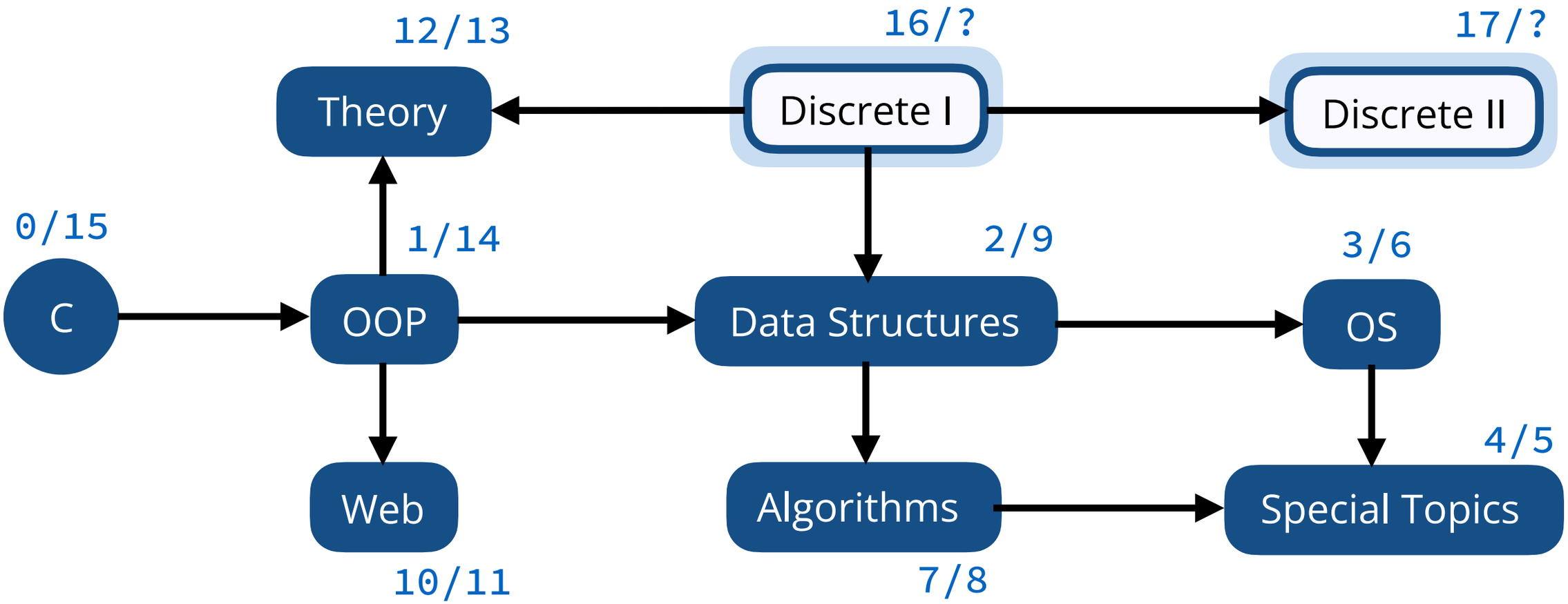
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



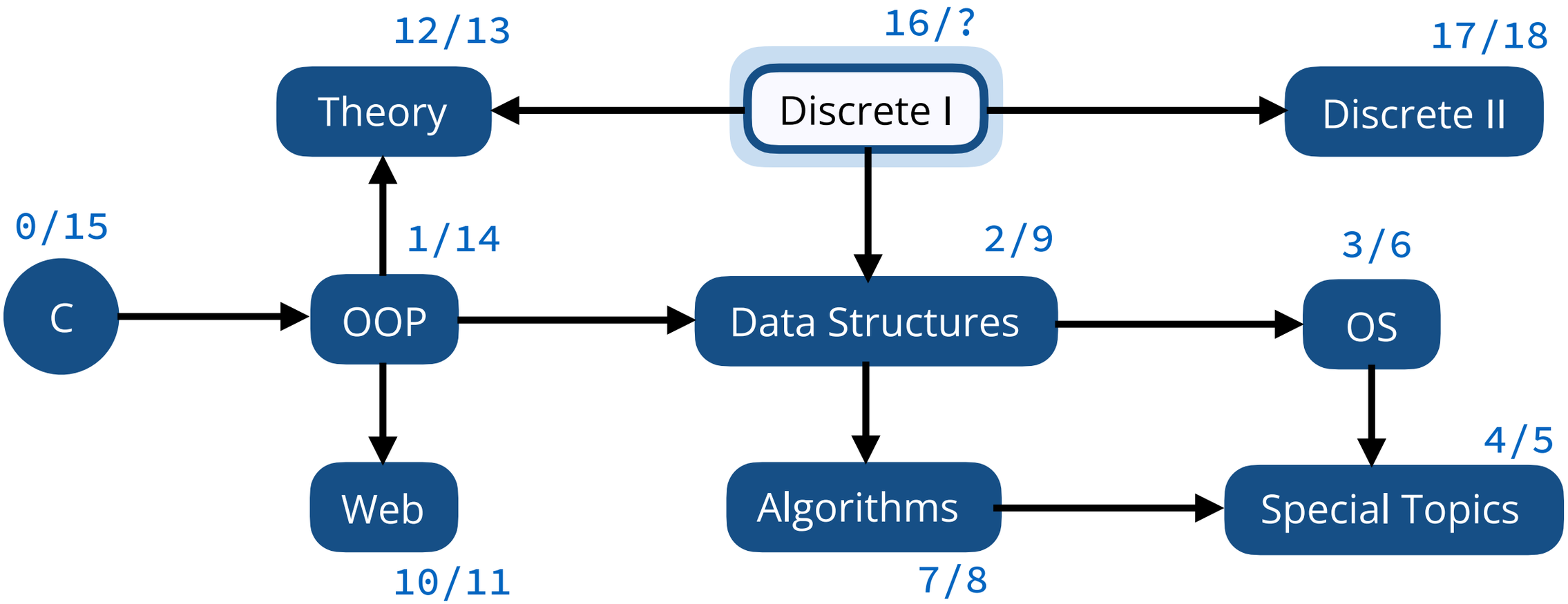
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



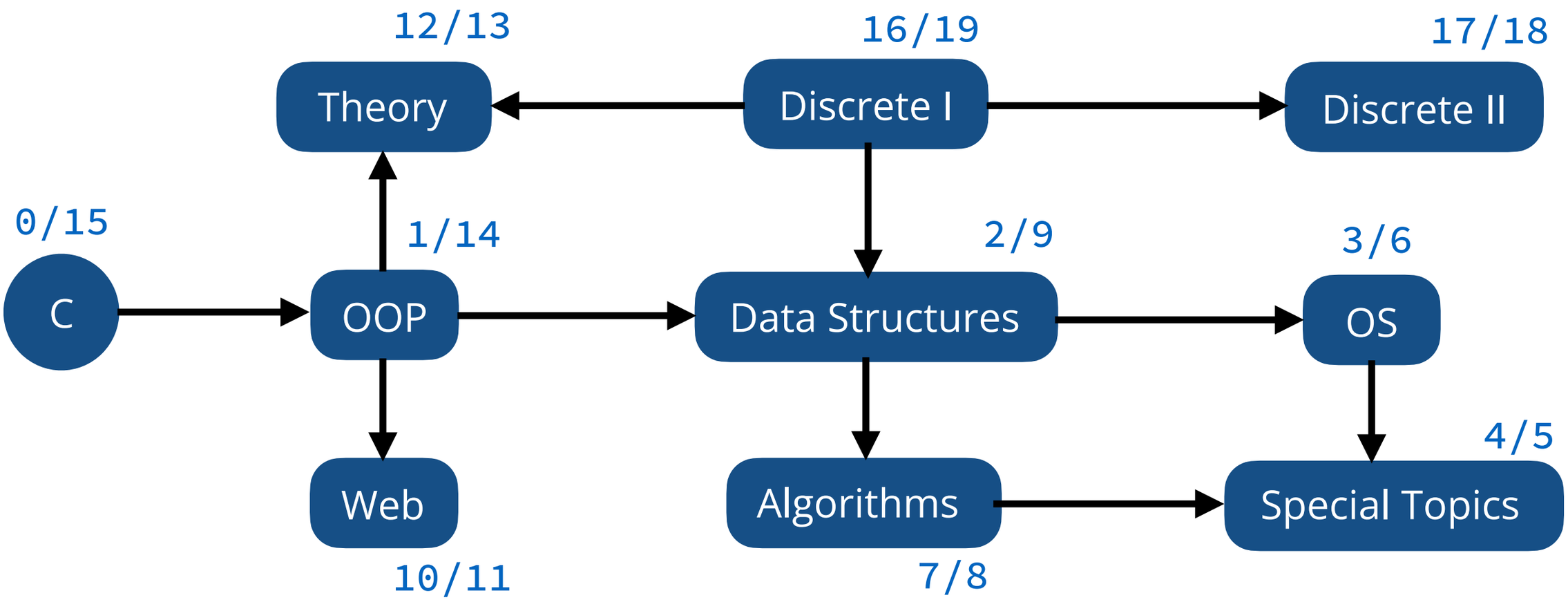
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).



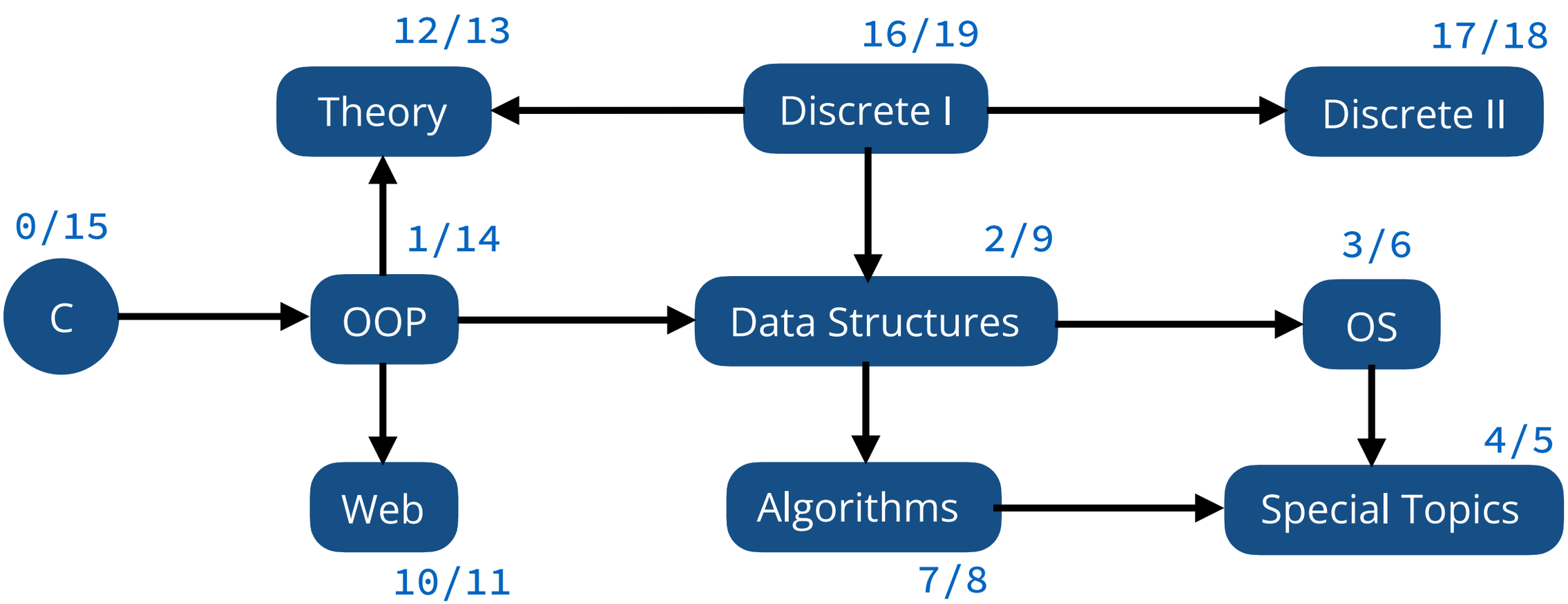
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).

Step 2. Report the vertices in *reverse finish time* order.

Discrete I → Discrete II → C → OOP → Theory → Web → Data Structures
f:19 f:18 f:15 f:14 f:13 f:11 f:9

→ Algorithms → OS → Special Topics
f:8 f:6 f:5



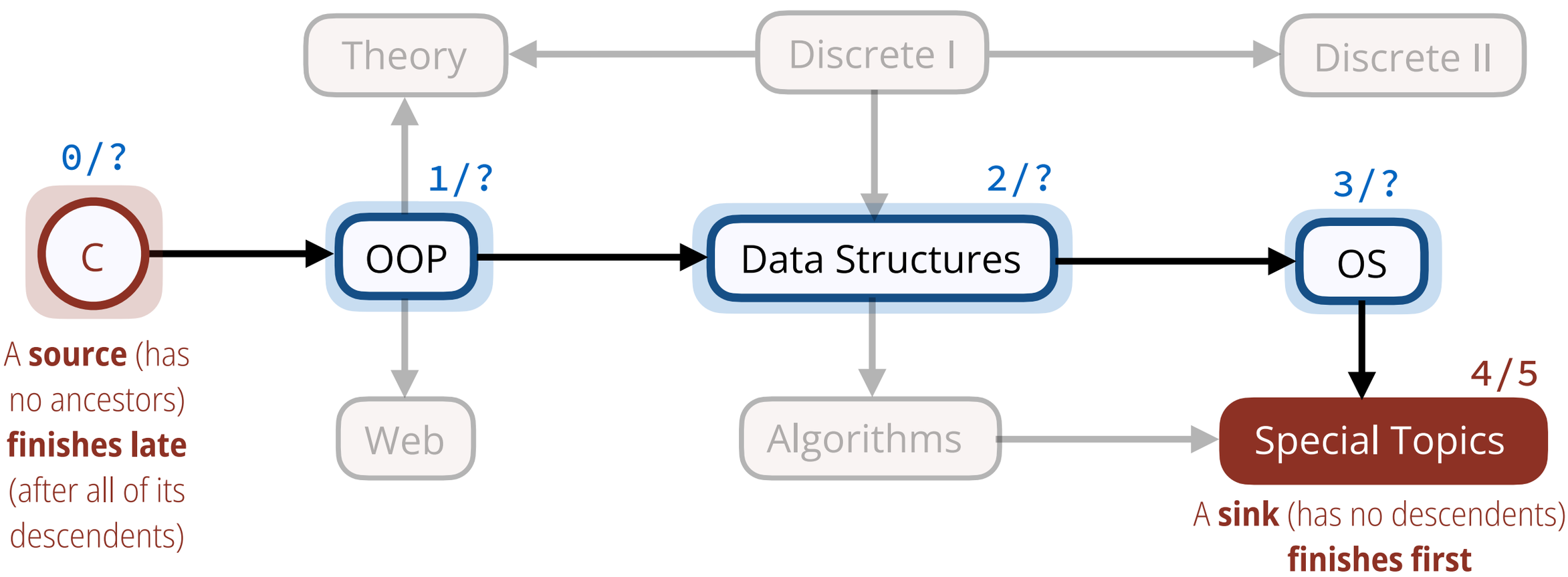
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).

Step 2. Report the vertices in *reverse finish time* order.

> Discrete I → Discrete II → **C** → OOP → Theory → Web → Data Structures → Algorithms → OS → **Special Topics**

Intuition. A vertex finishes after all of its descendants finish. I.e. a prerequisites finishes late but appears early in the topological order.



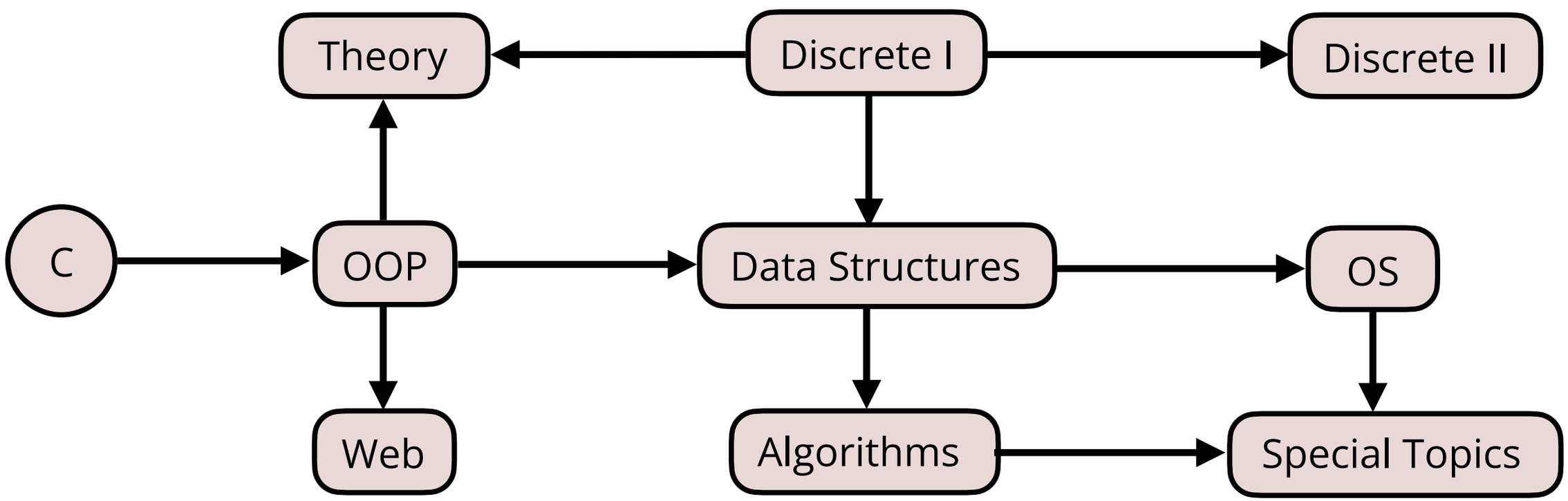
Topological Sort: Algorithm # 2

Step 1. Perform DFT to visit *all* vertices in the graph (it does not matter which is the source).

Step 2. Report the vertices in *reverse finish time* order.

Discrete I → Discrete II → C → OOP → Theory → Web → Data Structures → Algorithms → OS → Special Topics

Convince Yourself. Perform other DFTs using sources other than C and Discrete I. Check that the reverse finish time always gives a valid topological sort



Topological Sort: Implementation Sketch

```
TOPOLOGCAL-SORT(G)

  init color[] to WHITE
  define s as an empty stack

  for (every vertex v in G):
     if (color[v] == WHITE):
        DFT(v, color, s)

  while (!s.EMPTY())
     PRINT(s.POP())
```

PRINT the vertices in reverse finish time

```
DFT(v, color, s)

  color[v] = GRAY

  for (every w adjacent to v):
     if (color[w] == WHITE):
        DFT(w, color, s)

  color[v] = BLACK
  s.PUSH(v)
```

We can add an else statement to throw an error if `color[w] = GRAY` (i.e. if the graph has a cycle)

PUSH the vertex that finishes to a stack

Topological Sort: Implementation Sketch

```
TOPOLOGCAL-SORT(G)
  init color[] to WHITE
  define s as an empty stack

  for (every vertex v in G):
    if (color[v] == WHITE):
      DFT(v, color, s)

  while (!s.EMPTY())
    PRINT(s.POP())
```

PRINT the vertices in reverse finish time

```
DFT(v, color, s)
  color[v] = GRAY

  for (every w adjacent to v):
    if (color[w] == WHITE):
      DFT(w, color, s)

  color[v] = BLACK
  s.PUSH(v)
```

We can add an else statement to throw an error if `color[w] = GRAY` (i.e. if the graph has a cycle)

PUSH the vertex that finishes to a stack

Running Time.

Exactly the same as **TRAVERSE-ALL**(G)

$O(E + V)$ regardless of how the graph looks and what the source vertices are.

Exercise # 1

What is the maximum number of valid topological sorts in a DAG?

A. $V!$

B. 2^V

C. $O(V^2)$

D. $O(EV)$

E. E^V

D. $E^V \times V^E$

F. 42

Exercise # 1

What is the maximum number of valid topological sorts in a DAG?

A. $V!$

If the graph has no edges, there are $V!$ ways to order the vertices (no dependencies = all orderings are valid). Adding any edge adds a constraint and reduces the number of valid orderings.

B. 2^V

C. $O(V^2)$

D. $O(EV)$

E. E^V

D. $E^V \times V^E$

F. 42

Exercise # 2

What is the relationship between dynamic programming and topological sort?

Exercise # 2

What is the relationship between dynamic programming and topological sort?

Consider a dependency graph where each possible subproblem is a vertex in the graph, and there is an edge from u to v if subproblem v depends on subproblem u.
Bottom-up dynamic programming solves the subproblems in that graph in topological order!